A Data-Centric Perspective on the Lifecycle of Large Language Models

Jun Rao[1], **Xuebo Liu**[1†], **Haotian Yan**[1], **Junjie Shen**[1], **Haosi Mo**[1], **Yanghaopeng Dong**[1], **Zihao Yan**[1], **Ziyi Wang**[1], **Zhilin Li**[1], **Xiaojun Meng**[2†], **Zixiong Yu**[2], **Liqun Deng**[2], **Jiansheng Wei**[2], **Yunhe Wang**[3] **and Min Zhang**[1]

[1] Institute of Computing and Intelligence, Harbin Institute of Technology, Shenzhen, China
[2] Huawei Large Model Data Technology Lab. [3] Huawei Noah's Ark Lab.

[†] **Corresponding Authors.** ✉ liuxuebo@hit.edu.cn, xiaojun.meng@huawei.com

Abstract | This survey studies large language model (LLM) development through a data-centric lifecycle, with a particular focus on distinguishing what data artifacts are available, how they are transformed, and when and how they are consumed by the model. Rather than organizing the literature solely by training stage or individual technique, we introduce a three-tier framework comprising Data Substrates, Data Creation and Selection, and Data Ingestion Strategies. This decomposition provides explicit placement rules for methods that are otherwise difficult to categorize, such as model-aware selection, data mixing, executable agent environments, and test-time retrieval and verification. Based on a structured review of recent literature, we further synthesize several cross-tier trends: data value is increasingly model-dependent rather than universally defined by static quality scores; synthetic-data automation shifts human involvement from instance-level annotation toward specification and verification; agentic data is evolving from stored trajectories toward executable data-generating environments; and data optimization increasingly extends beyond training into adaptive inference-time information acquisition. We complement this synthesis with evidence-informed decision matrices that summarize setting-dependent trade-offs, failure modes, and applicability conditions. Together, the survey provides an operational framework for reasoning about how data artifacts, transformations, and consumption strategies jointly shape modern LLM development. This project can be found at https://github.com/raojay7/Awesome-LLMs-Data-AI.

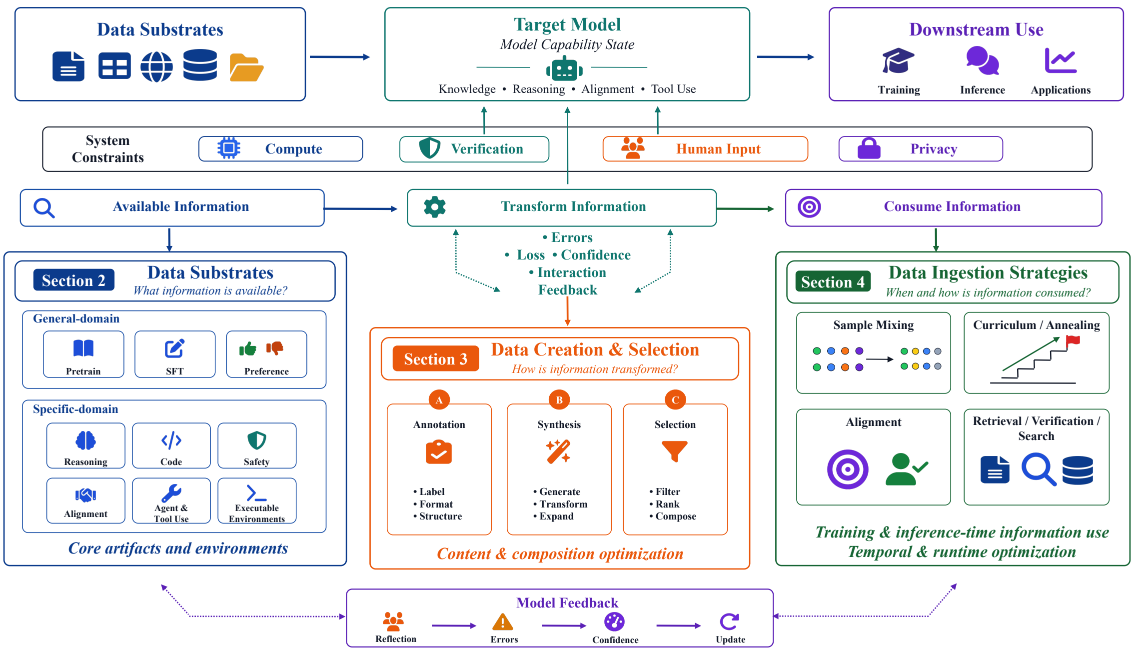

Figure 1 | Operational view of the LLM data lifecycle.

Contents

1. Introduction

The current wave of progress in artificial intelligence (AI) is no longer driven solely by architectural ingenuity or computational scale [Muennighoff et al., 2023, Vaswani et al., 2017]. Data also plays an increasingly important role. While the Transformer architecture has become widely adopted [Liu et al., 2024a, Zhu et al., 2024], differences across state-of-the-art large language models (LLMs) increasingly reflect not only model scale and optimization, but also the composition, filtering, and organization of their training data [Wei et al., 2022]. Yet, despite its central role, data remains the least theorized component of the LLM pipeline [Peng et al., 2023]. This survey aims to address the central role of data in LLM development by treating it not as a static resource [Chen et al., 2024a, Wang et al., 2022, Zhou et al., 2023] but as an active agent that shapes model intelligence. The emergence of the “data-centric” paradigm represents a fundamental shift from model-centric optimization [Muennighoff et al., 2023, Vaswani et al., 2017] to data epistemology: a discipline concerned with what knowledge is encoded, how it is structured, and how it evolves across training stages to sculpt artificial cognition. This perspective treats data as an active epistemic agent that co-evolves [Rao et al., 2025b] with the model, capable of inducing, refining, and even repairing capabilities through iterative optimization cycles. Rather than viewing data generation as an engineering task, filtering as quality control, or domain adaptation as a fine-tuning afterthought, we argue these processes must be unified under a theoretical framework that recognizes data as a dynamic extension of the model’s cognitive architecture. As illustrated in Figure 1, data permeates the lifecycle of LLMs.

Comparison with Existing Surveys Recent surveys have already provided substantial coverage of individual and overlapping components of the LLM data ecosystem. Data-selection surveys systematize criteria and algorithms for identifying useful training subsets Zha et al. [2023]; synthetic-data surveys Albalak et al. [2024] organize generation, augmentation, curation, and evaluation methods, in some cases across multiple stages of the LLM lifecycle Liu et al. [2024c], Long et al. [2024], Wang et al. [2024b], Zhou et al. [2025c]; broader LLM–data surveys additionally cover data processing, storage, serving, retrieval, and agentic workflows Zhang et al. [2026], Zhao et al. [2026b]. In parallel, recent agent surveys provide detailed taxonomies of tool learning Hu et al. [2026], agentic reasoning Wei et al. [2026], trajectory generation, and environment engineering Li et al. [2026a]. We therefore do not position this survey primarily on the basis of broader topical coverage. Instead, our distinction lies in the unit and axis of organization. Most existing frameworks are organized primarily around training stages, methodological families, data-management functions, or agent capabilities.

Our taxonomy asks a different set of operational questions: (1) what is the data-bearing artifact or environment, (2) what operation changes its content or inclusion, and (3) when and how is the resulting information consumed by the model? These questions correspond to Data Substrates, Data Creation and Selection, and Data Ingestion Strategies. The separation is particularly useful for methods whose placement is ambiguous in stage-based taxonomies. For example, offline data selection changes the composition of the data artifact and is therefore a creation/selection operation, whereas dynamic data mixing changes the weighting of samples over the optimization timeline and is an ingestion strategy. Likewise, an agent trajectory is a data substrate, an environment that generates trajectories is a data-producing substrate, trajectory synthesis is a creation operation, and test-time search is a runtime information-ingestion strategy. Accordingly, the purpose of the three-tier framework is not merely to provide another exhaustive inventory. It serves as an analytical device for comparing methods that operate at different levels of the data lifecycle and for identifying cross-tier transitions that are difficult to expose when synthesis, selection, training schedules, and agent environments are reviewed separately.

Survey Methodology We conducted a structured literature review designed to support both descriptive mapping of the field and cross-paper synthesis of recurring findings. Searches were conducted in

Table 1 | Positioning of representative surveys by their primary analytical scope and organizing principle. The comparison reflects the central organizing focus of each survey rather than whether a topic is mentioned somewhere in the paper. “Cross-tier placement rules” denotes an explicit distinction between data-bearing artifacts, operations that modify their content or inclusion, and strategies that modify their temporal or runtime consumption. Our goal is therefore not to claim uniformly broader topical coverage, but to highlight the different analytical decomposition adopted in this survey.

Representative Survey	Year	Primary Scope	Primary Organizing Principle	Cross-Stage Data Coverage	Executable Agentic Data	Artifact–Transformation –Ingestion Separation
Data-Centric AI [Zha et al., 2023]	2023	General data-centric AI	Data-centric workflow	Partial	No	No
Data Selection for LMs [Albalak et al., 2024]	2024	Data selection	Selection mechanisms/signals	Partial	No	No
LLM-Driven Synthetic Generation [Long et al., 2024]	2024	Synthetic generation and curation	Generation workflow	Partial	Limited	No
Data Synthesis & Augmentation [Wang et al., 2024b]	2024	Synthetic data across LLM stages	Training lifecycle	Yes	Limited	No
Survey of LLM DATA [Zhou et al., 2025c]	2025	Data processing, storage, and serving	Data-management functions	Yes	Yes	No
Agentic Tool Use in LLMs [Hu et al., 2026]	2026	Tool-using agents	Methodological evolution	Partial	Yes	No
Agentic Environment Engineering [Li et al., 2026a]	2026	Agent environments	Environment lifecycle	Partial	Yes	No
The LLM Data Auditor [Zhang et al., 2026]	2026	Data evaluation	Metric/auditing dimensions	Partial	Partial	No
Ours	**2026**	LLM data lifecycle	Object and operation being changed	Yes	Yes	Yes

Google Scholar, DBLP, and the ACM Digital Library, with the final search performed with a literature cut-off of June 2026. After consolidating duplicate publication versions, the final analytical corpus comprised 317 unique works. Query families covered four groups of terms: (i) data resources and training corpora, including pretraining corpora, instruction data, preference data, reasoning/code data, and agent trajectories; (ii) data creation, including annotation, instruction generation, knowledge distillation, synthetic-data generation, self-play, and environment/trajectory synthesis; (iii) data selection and curation, including quality filtering, diversity selection, influence- or gradient-based selection, and coreset construction; and (iv) data-use strategies, including data mixing, curriculum learning, data annealing, replay, retrieval, test-time search, and verification.

Backward and forward snowballing from representative works such as Self-Instruct, DCLM, and DoReMi was used to capture relevant papers missed by keyword search. We included studies that introduced a substantive data resource, data-construction or selection method, data-use strategy, or empirical analysis of how such interventions affect LLM training or inference. Short position pieces, non-technical commercial material, and works in which data played only an incidental role were excluded.

For taxonomy construction, the primary unit of coding was the artifact or operation being characterized, rather than the paper as a whole. Each work was coded by 3 reviewers for its primary lifecycle tier, secondary cross-cutting role where applicable, training stage, domain, intervention type, evaluation setting, and reported limitations. Disagreements were resolved by discussion. When a paper spanned several tiers, its primary label was assigned according to its main technical intervention, while secondary labels were retained to avoid forcing multi-stage methods into a single mutually exclusive category. Accordingly, a single method may contribute operations to multiple tiers. For example, a training-coupled data loop may generate or filter supervision in Tier 2 while regulating how the resulting supervision is consumed during optimization in Tier 3.

Importantly, we distinguish literature mapping from evidential synthesis. Statements describing the existence or taxonomy of a method may rely on an individual study, whereas broader claims in the summary sections are formulated only when a pattern recurs across multiple studies, model families, or application settings. Evidence from technical reports and closed industrial systems is used to document engineering practice but is not treated as controlled causal evidence. Where findings depend strongly on model scale, task distribution, verifier quality, proxy–target compatibility, or compute budget, we explicitly present them as conditional rather than universal. The decision matrices in Sections 2–4 should therefore be interpreted as evidence-informed design guidance, not as meta-analytic estimates of universally optimal configurations.

Taxonomy: An Operational Decomposition of the LLM Data Lifecycle Our taxonomy is designed to distinguish methods according to what component of the data lifecycle they directly modify. The resulting three tiers are therefore not simply consecutive training stages. Instead, they describe three operational roles.

Tier 1: Data Substrates (Artifacts and Environments) Data Substrates comprise the data-bearing artifacts and environments used across model development, adaptation, interaction, and evaluation, including corpora, instruction and preference examples, evaluation benchmarks, formal reasoning structures, interaction trajectories, and executable environments. Evaluation-only benchmarks are treated as observational substrates and are distinguished from data consumed for parameter optimization.

Tier 2: Data Creation and Selection (Content and Composition Transformations) Data Creation and Selection comprises operations whose primary effect is to create persistent new supervision or to modify the content, structure, labels, quality, diversity, or membership of existing data artifacts. Typical operations include annotation, synthesis, transformation, filtering, and sample selection.

Tier 3: Data Ingestion Strategies (Temporal Scheduling and Runtime Information Use) Data Ingestion Strategies comprises operations whose primary effect is to regulate how, when, how often, or under what contextual conditions available information or supervision is presented to and consumed by the model. This includes training-time sample organization and packing, contextual conditioning, mixture weighting, curriculum scheduling, annealing, replay, and runtime retrieval, search, decoding, and verification.

This distinction allows a single application area to span several tiers without becoming taxonomically ambiguous. Agentic learning, for example, may involve an executable environment as substrate, synthetic trajectory generation as creation, trajectory filtering as selection, and adaptive test-time interaction as ingestion. The same distinction separates offline sample selection from dynamic data mixing and separates stored retrieval corpora from runtime retrieval policies.

Taxonomy Placement Rules We use the direct object and dominant effect of an operation as the principal placement criterion. Operations whose primary effect is to create persistent new supervision or alter the content, labels, semantic structure, intrinsic properties, or membership of a data artifact are assigned primarily to Tier 2: Data Creation and Selection. In contrast, operations whose primary effect is to regulate how, when, how often, or under what contextual conditions available information is presented to and consumed by the model are assigned primarily to Tier 3: Data Ingestion Strategies. Tier 3 therefore includes not only temporal ordering and relative weighting, but also training-time sample organization, contextual conditioning, curriculum scheduling, replay, and runtime information acquisition.

This distinction can be summarized as follows: Tier 2 changes the supervision that exists, whereas Tier 3 changes how available supervision or information is delivered to the model. For example, offline data selection belongs primarily to Tier 2 because it changes which samples constitute the available dataset, whereas data mixing belongs to Tier 3 because it controls how those available samples are weighted during optimization. Similarly, generating or injecting a new negative response is a Tier-2 operation, while organizing existing positive and negative examples into an optimization schedule is a Tier-3 operation.

Some methods tightly couple these operations. For example, rejection-sampling fine-tuning may first generate and filter model-produced trajectories (Tier 2) and then immediately use the retained trajectories for iterative optimization (Tier 3). We therefore treat the operation rather than the paper as the primary unit of taxonomy, assigning primary and secondary tier labels when a method spans multiple stages. Data-bearing objects themselves, including corpora, trajectories, preference pairs, and executable environments, remain Tier-1 substrates.

Contributions of our Survey This survey makes four main contributions.

- **An operational lifecycle decomposition.** We distinguish data-bearing substrates, operations that change data morphology or membership, and strategies that regulate temporal or runtime information consumption. Explicit placement rules clarify boundary cases such as selection versus mixing, trajectory data versus trajectory generation, and retrieval corpora versus retrieval policies.
- **A cross-stage synthesis rather than a stage-by-stage inventory.** Applying the same decomposition across pretraining, post-training, reasoning, safety, and agentic systems reveals how similar data operations recur under different names and how the role of data changes as model capabilities evolve.
- **Four lifecycle-level trends grounded in the reviewed literature.** The synthesis highlights the shift from universal to model-dependent notions of useful data, from instance-level human annotation toward specification and verification in increasingly automated pipelines, from static agent traces toward executable data-generating environments, and from offline data preparation toward adaptive inference-time information acquisition.
- **Conditional engineering guidance.** Evidence-informed decision matrices summarize recurring applicability conditions, trade-offs, and failure modes. These matrices are intended as setting-dependent guidance rather than universal prescriptions, and the accompanying open resource records the literature underlying the taxonomy and synthesis.

2. Data Substrates: Core Artifacts and Executable Environments

To bridge abstract data taxonomies with actual industrial engineering, we analyze this substrate through the lenses of two illustrative model lineages that emphasize different aspects of data engineering: the **LLaMA lineage** (from LLaMA-1 to LLaMA-3.1) and the **DeepSeek lineage** (from DeepSeek-V2/V3 to DeepSeek-R1), as shown in Table 2. These lineages are used as illustrative case studies rather than as mutually exclusive paradigms, and the comparison is based on publicly disclosed information rather than controlled experiments. The LLaMA series exemplifies a web-centric, highly-filtered scaling paradigm focused on maximizing open-domain textual purity. Conversely, the DeepSeek series represents a multi-source, reasoning-centric paradigm that shifts the substrate early on toward verifiable symbolic structures and execution traces. Using these dual engineering anchors as catalysts, we classify and evaluate the data substrate across two macro-dimensions: General-Domain Substrates (§2.1) and Specific-Domain Substrates (§2.2).

2.1. General Domain

General-domain corpora supply the broad linguistic and world knowledge required by all subsequent specialisation steps. In Figure 3, each stage reconfigures the data-model boundary to build a specific layer of cognition: pretrain establishes a broad knowledge base, SFT refines task-specific behavior, and RL aligns with human values.

To systematically understand the data lifecycle of Large Language Models (LLMs), we must first isolate the foundational raw inputs, which we define as the **Data Substrate**. Rather than viewing data collection as a passive or linear preprocessing step, this section conceptualizes the substrate as the underlying empirical boundary and baseline capacity driver of all subsequent optimization phases.

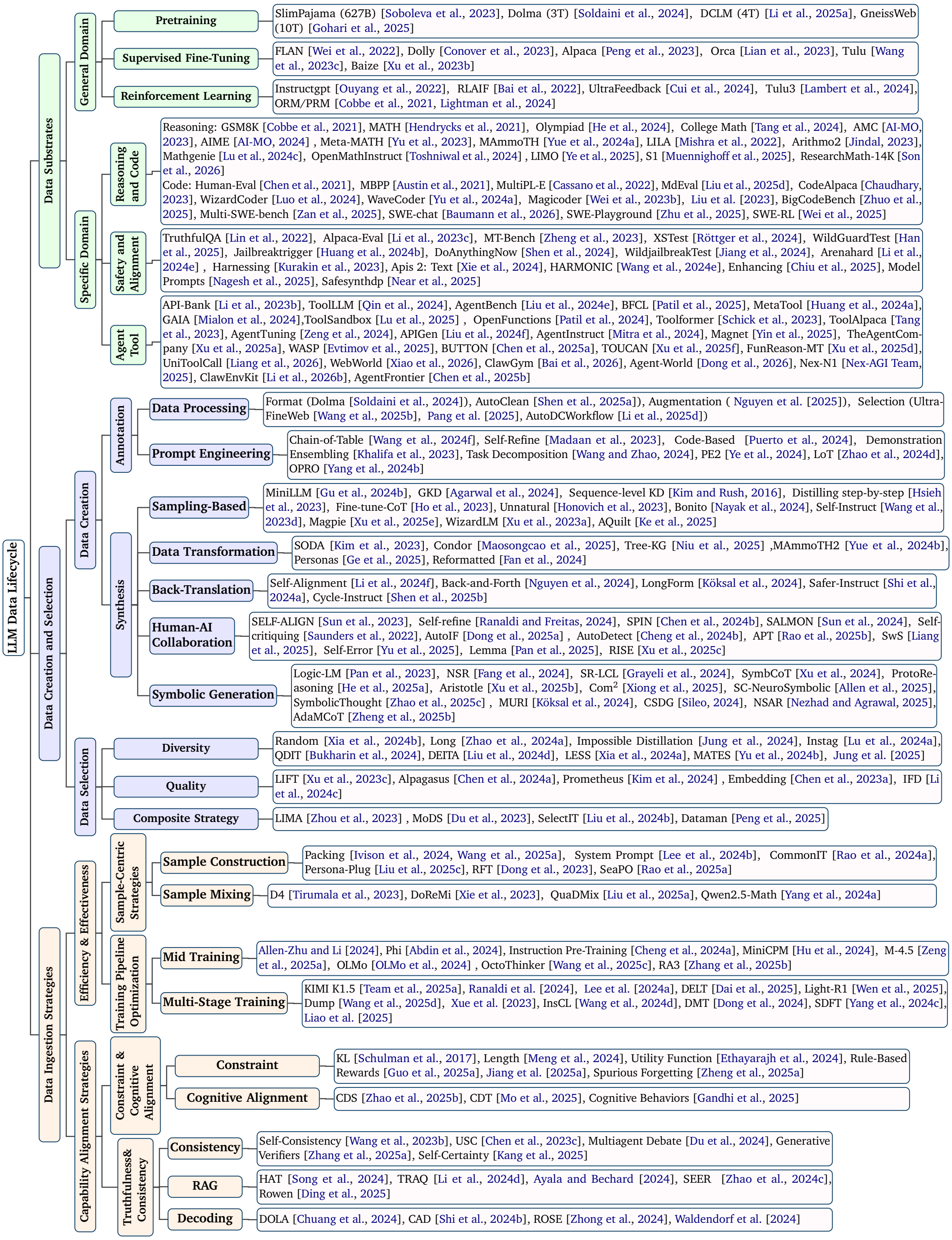

Figure 2 | Operational taxonomy of the LLM data lifecycle. We organize data-centric methods into three tiers according to what they directly act on: Data Substrates define the information-bearing artifacts and environments; Data Creation and Selection modifies their content or composition; and Data Ingestion Strategies determine when and how information is consumed during training or inference.

Table 2 | Anatomy of Data Engineering Paradigms: LLaMA vs. DeepSeek across the Three-Tier Lifecycle Framework.

Lifecycle Tier	**LLaMA Evolution Lineage (Web-Centric Baseline)**	**DeepSeek Evolution Lineage (Synthetic & Reasoning Frontier)**	**Abstract Methodology Mapping (§)**
Tier 1: Data Substrates (Artifacts and Environments)	Relies heavily on massive open-web crawl datasets (e.g., CommonCrawl, FineWeb). Focuses on scaling raw token volume while optimizing general common sense and multilingual semantic coverage.	Prioritizes deep integration of targeted, highly structural specific domains (e.g., source code, formal proofs), effectively converting static text blocks into executable infrastructure.	Section 2 • General Domain (§2.1) • Specific Domain (§2.2)
Tier 2: Data Creation and Selection (Content and Composition Transformations)	Employs extensive rule-based heuristic filtering, text-space deduplication, and black-box classification models for static pruning and intrinsic coreset extraction.	Leverages white-box parameter distillation, automated prompt seeding, autonomous self-evolving data flywheels, and execution-guaranteed symbolic verification.	Section 3 • Data Creation (§3.1) • Data Selection (§3.2)
Tier 3: Data Ingestion Strategies (Temporal Scheduling and Runtime Information Use)	Strictly follows a linear scheduling pipeline: uniform data mixing during early optimization, followed by a single-shot, high-concentration late-stage data annealing.	Implements multi-stage curriculum learning and model-aware dynamic mixture weights during training, while actively scaling data trajectory synthesis via test-time search.	Section 4 • Efficiency & Effectiveness (§4.1) • Capability Alignment (§4.2)

2.1.1. Pretraining

Pre-training substrates serve as the primary source of world knowledge and syntactic capabilities. The current engineering frontier is defined by the scaling of open-web crawl datasets (e.g., CommonCrawl, RefinedWeb, FineWeb) alongside highly-curated text repositories (e.g., books, Wikipedia, arXiv).

The LLaMA lineage illustrates the extreme limits of this paradigm. LLaMA-1 Touvron et al. [2023] initiated base training on 1.0–1.4 Trillion (T) tokens using heavily filtered open-source subsets. By the advent of LLaMA-3 Dubey et al. [2024], Meta expanded this physical corpus to 15T tokens, suggesting that larger and more carefully curated corpora can contribute to stronger foundation capabilities. However, model architecture, parameter scale, and training recipes also changed across generations, provided that raw text undergoes rigorous heuristic deduplication and model-based quality classification. This line of work confirms that general web text contains vast implicit world knowledge, but requires intensive spatial purification to prevent the model from optimizing over statistical noise.

Data here prioritizes scale (often billions of tokens) and diversity (covering multiple languages, genres, and domains) to ensure generalizability. Based on this approach, several pre-training datasets have seen rapid development. SlimPajama [Soboleva et al., 2023], a filtered version of RedPajama [Weber et al., 2024], is a commonly used pretraining corpus, which consists of seven subcorpora, including CommonCrawl, C4, Wikipedia, GitHub, StackExchange, ArXiv, and Book. Most of these existing datasets are already available on the internet, and models are trained on heterogeneous data spanning many domains, sources, and formats. Yang et al. [2025] used a small-scale domain-specific corpus to synthesize a larger corpus that was more suitable for learning, and then conducted continuous pre-training on the synthesized corpus. It demonstrated how synthetic data augmentation

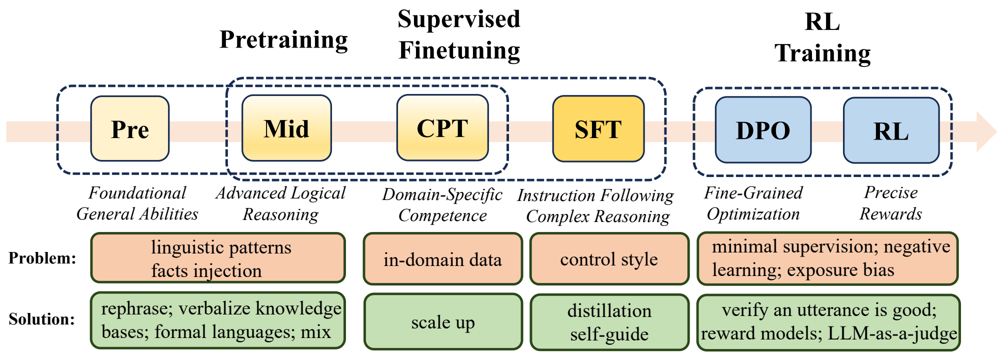

Figure 3 | The language modeling pipeline. Each stage has distinct objectives and data construction.

enables more efficient data learning by "reorganizing" knowledge. Different organizations have also collected pre-trained data of varying sizes through unique filtering operations [Gohari et al., 2025, Li et al., 2025a]. Essential-Web v1.0 [AI, 2025] is a 24-trillion-token dataset where all documents are annotated with a twelve-category taxonomy covering topic, format, content complexity, and quality.

2.1.2. SFT

Once a base model is pretrained, it requires instruction-following data to bridge the gap between text-completion and user intent alignment. Traditional general-domain SFT corpora include templated NLP task-conversions (e.g., FLAN, Super-Natural Instructions) and crowd-sourced multi-turn human conversations (e.g., ShareGPT).

The operational boundary of these general SFT substrates is heavily theorized by the *Surface Alignment Hypothesis*. Evidence from curated low-volume collections such as LIMA suggests that high-quality examples can establish basic instruction-following behavior with relatively limited data. Beyond this point, scaling generic conversational SFT data may yield diminishing returns and may strengthen response style more than task-specific knowledge or reasoning. SFT optimizes the model to minimize loss on task-specific labeled data. Early instruction data was typically derived through simple conversion or distillation. FLAN 2021 [Wei et al., 2022] and Super-Natural Instructions [Wang et al., 2022] involve converting traditional NLP tasks into an instruction format using manually defined instruction templates. Wang et al. [2023c] and Lambert et al. [2024] collected a comprehensive and representative sample of datasets across various styles called Tulu mix.

2.1.3. RL

RL further aligns the model with human values, safety constraints, or user preferences, representing the ethical cognition engineering phase. This stage uses feedback signals (human ratings, automated rewards) to optimize behavior beyond what SFT alone can achieve. Data here consists of feedback scores or rankings (e.g., human ratings of response quality, safety, helpfulness) that guide the model to prioritize desirable cognitive outputs.

RL optimizes the model using a reward function $r(\cdot)$ derived from feedback data. Using proximal policy optimization (PPO) [Schulman et al., 2017], a common algorithm in RL for LLMs, the objective includes a policy loss and a constraint to avoid large deviations from the SFT model. DPO [Rafailov et al., 2023] directly optimizes the model using preference data (pairs of responses where one is preferred over the other) without explicitly training a reward model or using a reinforcement learning loop, simplifying the alignment process. Most of the datasets at this stage are annotated by humans [Ouyang et al., 2022] or judged by reward models [Cui et al., 2024, Dubois et al., 2024] or automated validators [Lightman et al., 2024]. RL from AI feedback (large models [Cui et al., 2024, Lambert et al., 2024] or self [Chen et al., 2024b]) is the mainstream method for creating RL datasets due to considerations of cost and effectiveness. RLAIF [Bai et al., 2022] trains preference models with AI-generated feedback from constitutions. SELF-ALIGN [Sun et al., 2023] minimizes human supervision for training AI-assistants by using synthetic prompts and human principles to guide learning, improving response quality. SALMON [Sun et al., 2024] introduces an AI alignment paradigm where an instructable reward model is trained to effectively and flexibly align language models with human values and intentions.

2.2. Specific Domain

As LLMs transition from conversational fluency to expert-level system execution, general web text encounters a strict capability ceiling. Specific-domain substrates introduce specialized, highly structural tokens that anchor statistical distributions in formal logic, safety boundaries, and environment dynamics.

2.2.1. Reasoning and Code

Reasoning substrates differ fundamentally from standard text by encoding deterministic structural dependencies. These assets comprise formal mathematical derivations (e.g., MATH, GSM8K, Open-WebMath) and functional code repositories (e.g., Stack-V2, GitHub commits, unit-test suites). We have presented their comparison in an intuitive manner, as shown in Table 3, Table 4, Table 5 and Table 6. The DeepSeek Zhu et al. [2024] lineage directly showcases the strategic primacy of this substrate. In DeepSeek-Coder-V2 and DeepSeek-V3, code and mathematical structures were not treated as mere fine-tuning add-ons but were synthesized directly into the foundational pretraining mixture at an unprecedented ratio. Code substrates introduce highly strict syntax boundaries and logical causality to the data mixture. They can provide structured supervision for learning long-horizon dependencies and may facilitate subsequent reasoning-oriented post-training.

Classic Dataset: In the early days, there were many datasets suitable for mathematical arithmetic reasoning, such as MAWPS [Koncel-Kedziorski et al., 2016], AQUA-RAT [Ling et al., 2017], GSM8K [Cobbe et al., 2021], MATH [Hendrycks et al., 2021], Gaokao 2023 En [Liao et al., 2024], Olympiad Bench [He et al., 2024], and College Math [Tang et al., 2024]. In addition, some tasks are used to test different dimensions of mathematics, such as system-level mathematical tests [Mishra et al., 2022, Zheng et al., 2022], geometric problems [Chen et al., 2022, Masry et al., 2022], visual content [Lu et al., 2024b], theoretical proofs [Chen et al., 2023b, Yang et al., 2023], and Chinese mathematics [Wei et al., 2023a]. In recent years, with the improvement of the model's mathematical capabilities, difficult competition-level tests have also been added to the model's tests, such as AMC 23 [AI-MO, 2023], and AIME 24 [AI-MO, 2024]. ResearchMath-14K extends this curated-data direction with agent-assisted extraction and refinement: it turns research-level mathematical sources into self-contained problems and verifies them with status auditing and deduplication [Son et al., 2026].

Early and existing efforts in evaluating LLMs' code-related capabilities have primarily focused on three aspects: 1) basic code generation, such as HumanEval [Chen et al., 2021], MBPP [Austin et al., 2021], and MultiPL-E [Cassano et al., 2022]; 2) code editing and repair, such as Aider Benchmark [AI, 2023], MdEval [Liu et al., 2025d], and 3) text-to-SQL [Yu et al., 2018]. With the growing demand for LLM applications in engineering, many recent datasets focus on evaluating repository-level code engineering capabilities, such as HumanEval-FIM [Allal et al., 2023], CrossCodeEval [Ding et al.,

Table 3 | Curated mathematical benchmark datasets. Task Domain abbreviations: (1) arithmetic word reasoning, (2) advanced/competition math, (3) formal theorem proving, (4) geometry/visual math, (5) multi-task/system-level math; ✓ indicates covered capability.

Dataset	**Year**	**Size**	**Task Domain**	**Eval. Metric**
MAWPS	2016	1.7k	1✓	Acc
AQUA-RAT	2017	196k	1✓	Acc, BLEU, ROUGE
GSM8K	2021	8.8k	1✓	Acc
MATH	2021	12.5k	2✓	Acc
Minif2f	2022	488	3✓	Proof Success, Pass@K
LILA	2022	~134k	5✓	Acc, F1
UniGeo	2022	14.5k	4✓	Acc
ChartQA	2022	32.7k	4✓	Relaxed Acc, Exact Match
ProofNet	2023	371	3✓	BLEU, Proof Validity
TheoremQA	2023	800	3✓	Acc, Retrieval Acc
LeanDojo	2023	98.7k	3✓	Pass@1
Gaokao-Bench	2023	2.8k	2✓	Acc
AMC 23	2023	40	2✓	Acc
Gaokao 2023 En	2024	385	2✓	Acc
Cmath	2023	1.7k	1✓	Acc
Olympiad Bench	2024	8.5k	2✓	Acc, Pass@K
College Math	2024	3.7k	2✓	Acc
MathVista	2024	6.1k	4✓	Acc, Visual Score
AIME 24	2024	30	2✓	Acc
ResearchMath-14K	2026	14k; 220k traj.	2✓ 3✓ 5✓	Pass@K, Verifier Acc

Table 4 | Synthetic and instruction-tuning mathematical datasets. Column definitions are consistent with Table 3.

Dataset	Year	Size	Task Domain	Eval. Metric
Arithmo2	2023	~540k	1✓	Pass@1
MetaMathQA	2023	395k pairs	1✓	Acc
MMIQC	2024	2.3M pairs	1✓	Acc
MuggleMath	2024	500	1✓	Acc, Pass@1
MathGenie	2024	284k pairs	1✓	Pass@K
OpenMathInstruct-1	2024	1.8M pairs	5✓	Pass@1
MathInstruct	2024	262k samples	5✓	Acc
MegaMath	2025	217B tokens	5✓	Pre-training corpus
DeepMath-103K	2025	103k	2✓	Pass@1
S1	2025	1k	1✓	Pass@1
LIMO	2025	817	5✓	Acc

2023], CrossCodeLongEval [Wu et al., 2024], RepoEval [Zhang et al., 2023]. Recent resources further extend code evaluation to realistic repository and user interaction settings. Multi-SWE-bench contains human-validated issue-resolving tasks from repositories in eight programming languages and provides reproducible execution environments [Zan et al., 2025]. SWE-chat instead collects real coding-agent sessions with user prompts, tool calls, checkpoints, code changes, and commits, making it possible to study which agent outputs users retain, revise, or reject [Baumann et al., 2026]. These resources are therefore better viewed as curated evaluation or interaction datasets rather than synthetic code data.

Synthetic Dataset: With the development of synthetic data, the mathematical capabilities of LLMs are growing rapidly [Azerbayev et al., 2024]. Early mathematical data was expanded and filtered using rules. Arithmo2 [Jindal, 2023] combined existing math instruction dataset like MetaMath [Yu et al., 2023] , MAmmoTH [Yue et al., 2024a] and LILA [Mishra et al., 2022]. Recent work has used LLMs for further data expansion from existing datasets. MuggleMath [Li et al., 2024a], Mathgenie [Lu et al., 2024c], and Liu et al. [2025b] generated data based on the seeds and performed data augmentation. Some work also utilizes LLMs to expand the scale of data [He et al., 2025d, Toshniwal et al., 2024, Zhou et al., 2025a]. Some studies use a small amount of high-quality long CoT data to stimulate models and obtain good mathematical reasoning test results, such as S1 [Muennighoff et al., 2025] and LIMO [Ye et al., 2025]. Its companion generated supervision further releases teacher reasoning trajectories, which makes ResearchMath a hybrid case: curated research problems supply the base data, while agent-produced trajectories provide synthetic training signals [Son et al., 2026].

In code-related tasks, current work primarily utilizes methods like Self-Instruct or Evol-Instruct to create large-scale, high-quality instruction-response pairs, such as Code Alpaca [Chaudhary, 2023] and WizardCoder [Luo et al., 2024]. Additionally, some efforts focus on multi-task fine-tuning, covering code generation, completion, and debugging tasks, as seen in WaveCoder [Yu et al., 2024a] and Magicoder [Wei et al., 2023b]. Some evaluation efforts primarily utilize LLMs to augment data. For instance, Liu et al. [2023] creates HumanEval+ and MBPP+ with expanding data using seed inputs and mutation algorithms. BigCodeBench [Zhuo et al., 2025] generates core tasks with LLMs, refined through human feedback. CRUXEval [Gu et al., 2024a] relies entirely on LLM-generated functions. Some works combine real-world data collection with LLM-driven data augmentation (LiveCodeBench [Jain et al., 2025], McEval [Chai et al., 2025], and BIRD [Li et al., 2023a]). For repository-level agent training, SWE-Playground synthesizes complete software projects, engineering tasks, test suites, and executable coding-agent trajectories without relying on existing issues [Zhu et al., 2025]. SWE-ZERO/SWE-HERO similarly bootstraps executable tasks and successful trajectories from repository context and iteratively couples task generation with task solving [Zhao et al., 2026a]. In contrast, SWE-RL mines naturally occurring issues, pull requests, code snapshots, and patches into reinforcement-learning supervision from open software evolution, so it belongs to real-world data construction rather than pure synthesis [Wei et al., 2025].

Table 5 | Curated/real-world code datasets. Task Domain abbreviations: 1=Code Generation, 2=Code Reasoning, 3=Code Edit/Repair, 4=Text-to-SQL, 5=Repo-level Completion; ✓ = dataset covers the capability.

Dataset	Year	Size	Task Domain	Eval Metric
Spider	2018	10k+5.7k	4✓	Exec Acc, Exact Match
HumanEval	2021	164	1✓	Pass@K
MBPP	2021	974	1✓	Pass@K
MultiPL-E	2022	2k+	1✓	Pass@K
The Stack V1.1	2022	545M	1✓	Pass@K
Aider Benchmark	2023	307	3✓	Pass@1, Pass rate
CrossCodeEval	2023	10k	5✓	Exact Match, Edit Similarity, F1
RepoEval	2023	15 repos	5✓	Unit Test Pass
CrossCodeLongEval	2024	1.5k repos	5✓	Exact Match, Edit Similarity
MdEval	2025	3.9k	3✓	Acc, Pass@1
Multi-SWE-bench	2025	1.6k issues	3✓ 5✓	Resolved Rate, Pass-to-Pass
SWE-RL	2025	1.6k issues	3✓ 5✓	Resolve Rate, RL Reward
SWE-chat	2026	Real sessions	3✓ 5✓	User Retention, Commit Signal

2.2.2. *Safety and Alignment*

Safety substrates are highly specialized contrastive datasets designed to establish a model's refusal boundaries. These datasets consist of adversarial prompts (red-teaming vectors, jailbreak templates) mapped to standard, non-compliant safety responses (e.g., HarmBench, AdvGLUE). Unlike general text, safety substrates must maintain extreme semantic precision: over-indexing on uniform refusal substrates causes the model's decision boundaries to collapse, leading to acute over-refusal syndromes

Table 6 | Synthetic/instruction-tuning code datasets. Column definitions consistent with Table 5.

Dataset	Year	Size	Task Domain	Eval Metric
Code Alpaca	2023	20k pairs	1✓	BLEU, Pass@K
HumanEval+	2023	164	1✓	Pass@K, Functional Correctness
MBPP+	2023	378	1✓	Pass@K, Functional
WizardLM Evol Instruct	2024	70k	1✓	Pass@K, BLEU, Exec acc
Magicoder	2024	75k pairs	1✓	Pass@K, CodeBLEU, Exec acc
CodeSeaXDataset	2024	20k instances	1✓ 3✓	Pass@K, Debug Success Rate
LiveCodeBench	2024	400	1✓	Pass@1, Self-Repair Rate, Exec Acc
CRUXEval	2024	800	2✓	Pass@1, Inference Acc, CoT Improvement
BigCodeBench	2025	1.1k tasks	1✓	Pass@K, Functional Correctness, Test Coverage
McEval	2025	16k samples	1✓	Pass@1, Acc, Cross-Language Perf
SWE-Playground	2025	Synthetic repos	3✓ 5✓	Resolve Rate, Test Pass Rate

Table 7 | Representative open-ended model evaluation benchmarks. Task abbreviations: (1) truthful question generation, (2) general instruction following, (3) multi-turn open-ended chat; ✓ indicates covered capability.

Dataset	Year	Size	Task	Metrics
TruthfulQA	2022	817 questions	1✓	Generation: truthfulness, informativeness; MC: MC1, MC2
AlpacaEval	2023	805 instructions	2✓	Win Rate
MT-Bench	2023	80 instructions	3✓	Total Score
ArenaHard	2024	500 prompts	2✓	Win Rate

where entirely benign user requests containing sensitive trigger words are falsely rejected. We have presented the classic datasets in an intuitive manner, as shown in Table 8 and Table 7.

Classic Dataset: The development of safety evaluation dataset has significantly advanced the study of LLM robustness and misuse prevention. Recent works primarily design adversarial testing frameworks to probe models under safety-critical scenarios. HarmBench [Mazeika et al., 2024] systematically quantifies harmful content generation risks. For example, XSTest is a test set used to detect model over-safety and over-rejection [Röttger et al., 2024], while WildGuardTest investigates model responses in real-world harmful contexts [Han et al., 2025]. Similarly, JailbreakTrigger [Huang et al., 2024b] and DoAnythingNow [Shen et al., 2024] create prompt-based attack suites to expose hidden unsafe behaviors. WildJailbreakTest [Jiang et al., 2024] extends these methods by simulating collaborative adversarial teaming in uncontrolled environments.

In parallel, alignment benchmarks aim to measure how well LLMs follow truthful, helpful, and harmless principles. TruthfulQA, for instance, assesses factual consistency by detecting hallucinations and misinformation [Lin et al., 2022]. AlpacaEval provides a lightweight benchmark for instruction-following comparisons [Li et al., 2023c], while MT-Bench introduces multi-turn conversational evaluations for open-ended alignment [Zheng et al., 2023]. More recently, ArenaHard proposes a large-scale arena-based evaluation with challenging instructions [Li et al., 2024e]. Taken together, these benchmarks form a comprehensive framework for investigating safety and alignment in synthetic data generation and LLM training.

Table 8 | Comparison of classic safety datasets. Task abbreviations: (1) harmful query safeguard, (2) toxic generation, (3) balance between refusal & over-refusal, (4) over-refusal on benign queries, (5) templated jailbreak defense, (6) human-written jailbreak defense; ✓ indicates covered capability.

Dataset	Year	Size	Task	Metrics
TOXIGEN	2022	274k statements	1✓	Toxicity
HarmBench	2024	18 red team methods	2✓	Attack Success Rate
XSTest	2024	450 prompts	3✓ 4✓	Refusal to Answer, F1
JailbreakTrigger	2024	13 attack methods	6✓	Refusal to Answer
DoAnythingNow	2024	107k samples	6✓	Attack Success Rate
WildJailbreak	2024	262k pairs	3✓ 4✓	Attack Success Rate, Query Times, Perplexity
WildGuardMix	2024	88,484 examples	1✓ 3✓	Accuracy (Harm/Refusal)

Synthetic Dataset: The cognitive challenge lies in translating multifaceted human values into consistent training signals. Effective safety datasets must encode not merely rules or restrictions, but the underlying reasoning processes that humans use to navigate ethical dilemmas. This requires data that captures the nuanced interplay between context, consequences, and principles: enabling models to develop ethical reasoning capabilities rather than superficial rule-following behaviors. Early studies, such as Kurakin et al. [2023], explore generating private synthetic text through carefully designed prompts, reducing the risk of direct memorization. Building on this line, recent work has increasingly integrated differential privacy mechanisms, for example, SafeSynthDP [Nahid and Hasan, 2024] and DP-based foundation model APIs [Xie et al., 2024], which inject noise during the generation process to mitigate leakage while preserving downstream utility. Beyond unstructured text, Wang et al.

Table 9 | Curated agent and tool-use benchmark datasets. Task abbreviations: (1) function calling, (2) tool selection, (3) multi-step planning, (4) interactive/environment dialogue, (5) GUI/embodied interaction; ✓ denotes covered capability.

Dataset	Year	Size	Task	Metrics
API-Bank	2023	314 dialogues; 753 API calls	1✓	Acc, ROUGE-L
MetaTool	2023	21k tool-selection queries	2✓	Acc, Correct Selection Rate
AgentBench	2023	8 environments; ~1.3k samples	4✓	Success Rate, F1, Reward
GAIA	2023	466 questions	4✓	Acc
WebArena	2023	812 long-horizon web tasks	4✓	Task Success Rate, Exact Match
ToolSandbox	2024	1,032 conversational tool-use cases	4✓	Milestone Similarity, Turn Count
BFCL	2025	5.5k question-function-answer pairs	1✓	Acc, AST Match, Exact Match
T1	2025	13.5k tool-use dialogues	4✓	F1, Code Exec Rate, Exact Match
WebWorld	2026	Web interaction trajectories	3✓ 4✓ 5✓	Task Success, Trajectory Quality
ClawGym	2026	Persona tasks, mock workspaces	1✓ 2✓ 3✓ 4✓ 5✓	Success Rate, Hybrid Verification
UniToolCall	2026	Mixed real/synthetic tool data	1✓ 2✓ 3✓ 4✓	Acc, Sandbox Success

[2024e] introduce HARMONIC, a framework that extends LLM-driven synthesis to tabular data and incorporates privacy-aware evaluation. More recently, Nagesh et al. [2025] advances prompt seeding techniques to ensure utility-preserving private text generation, particularly in sensitive domains such as healthcare. However, synthetic data is not without risks. Chiu et al. [2025] demonstrate that LLM-generated samples can also be exploited to enhance leakage attacks against encrypted systems, underscoring the dual-use nature of synthetic datasets and the need for robust safeguards.

2.2.3. *Agent and Tool Use*

As shown in Table 9 and Table 10, LLMs learn to extend their reasoning beyond internal parameters by leveraging external resources (APIs, databases, tools). The frontier of data engineering shifts the substrate from a passive text document into an active, programmable environment trace. Agentic substrates record the multi-turn interaction loops between an LLM, external tools (APIs, calculators), and sandboxed software environments.

Early frameworks relied on static, human-logged API history arrays (e.g., API-Bank, ToolLLM). However, the development of modern coding and automation agents demonstrates that static logs introduce severe exposure bias. Consequently, the contemporary state-of-the-art utilizes executable, environment-grounded tracking systems (e.g., Multi-SWE-bench, TOUCAN, ClawGym). In these setups, the data substrate consists of dynamic execution trajectories, terminal outputs, and automated compiler feedback loops, representing a complete evolution of data assets from offline textual artifacts to executable cybernetic workflows.

Classic Dataset: Early datasets for agent tool use were established to evaluate fundamental capabilities, such as API planning and invocation across diverse scenarios, including API-Bank [Li

Table 10 | Synthetic agent and tool-use datasets. Column definitions are consistent with Table 9.

Dataset	Year	Size	Task	Metrics
Toolformer	2023	Samples for 5 tools	1✓	Perplexity, Zero-shot Acc, Task Acc
Gorilla (APIBench)	2023	API instruction dataset	1✓	Accuracy, Hallucination, AST-matching
ToolLLM	2023	126k tool-use pairs	2✓	Pass Rate, Win Rate
ToolAlpaca	2023	3.9k cases; 426 tools	2✓	Acc
AgentTuning	2023	1.8k trajectories	3✓	Success Rate, F1, Reward
APIGen	2024	60k samples; 3,673 APIs	1✓	Acc, Pass@K
AgentInstruct	2024	Millions of agent flows	3✓	Acc, Pass@K, Visual Score
CodeActInstruct	2024	7k agent-environment trajectories	5✓	Success Rate, Avg Turns, Completion
APIGen-MT	2025	5k multi-turn dialogues	3✓	Pass@1, Acc, Consistency, Hallucination
Magnet	2025	34k SFT + 4.5k preference pairs	3✓	Pass@1, Acc, Multi-turn Success
BUTTON	2025	Multi-turn function-calling data	1✓ 3✓ 4✓	Success Rate, Exact Match
TOUCAN	2025	1.5M MCP-grounded trajectories	1✓ 2✓ 3✓ 4✓	Exec Success, Coverage, Completeness
FunReason-MT	2025	Environment-API multi-turn traces	1✓ 3✓ 4✓	BFCL Acc, Pass Rate
AgentFrontier	2025	ZPD-guided synthetic data	2✓ 3✓ 4✓	Success Rate, Capability Gain
Nex-N1	2025	Grounded environment trajectories	1✓ 2✓ 3✓ 4✓ 5✓	Task Success, Pass Rate
Agent-World	2026	Synthesized tool ecosystems	1✓ 2✓ 3✓ 4✓	Task Success, Capability Gap
ClawEnvKit	2026	Generated tasks and scorers	1✓ 2✓ 3✓ 4✓ 5✓	Env Success, Score Validity

et al., 2023b], the large-scale ToolLLM [Qin et al., 2024], and the multi-environment AgentBench [Liu et al., 2024e]. In addition, various benchmarks were proposed to test different dimensions of agentic behavior, such as interacting with realistic web environments [Zhou et al., 2024], the critical decision-making process of tool selection [Huang et al., 2024a], and solving general assistant tasks that require a combination of tool use and reasoning [Mialon et al., 2024]. More recently, as models have become more capable, the focus has shifted towards more complex and fine-grained evaluations, including benchmarks designed for stateful and conversational interactions [Chakraborty et al., 2025, Lu et al., 2025], the underlying mechanics of function calling [Patil et al., 2025], and multi-step reasoning in specialized domains like data analysis [Egg et al., 2025]. TheAgentCompany extends evaluation to a simulated software company in which agents must browse, code, communicate, and coordinate over long-horizon workplace tasks [Xu et al., 2025a]. Security-oriented interaction data are also becoming important: WASP injects adversarial content into dynamic web environments and separately measures whether the user objective is disrupted and whether the attacker's goal succeeds [Evtimov et al., 2025].

Synthetic Dataset: Agent and tool use datasets encode cognitive offloading strategies, teaching models to decide which tasks to handle internally and which to delegate to tools: Early work like Toolformer [Schick et al., 2023] established that LLMs could teach themselves to use tools, inspiring self-instruction methods where powerful teacher models generate foundational datasets for works like ToolAlpaca [Tang et al., 2023] and AgentTuning [Zeng et al., 2024]. Recent efforts have focused on improving data quality and verifiability. For instance, APIGen introduced automated pipelines with execution-based checks [Liu et al., 2024f], while OpenFunctions ensures API calls remain accurate [Patil et al., 2024]. To enable more sophisticated capabilities, data synthesis is increasingly targeting complex, multi-step agent flows [Mitra et al., 2024] and multi-turn interactions [Yin et al., 2025]. This now also includes generating interaction trajectories within stateful environments, creating datasets that teach agents to perform complex tasks like computer operations [He et al., 2025b, Wang et al., 2024c]. To streamline these diverse methods, frameworks like DataDreamer have emerged to offer reproducible and modular synthetic data generation workflows [Patel et al., 2024]. For multi-turn function calling, BUTTON composes atomic real-world tasks and simulates user, assistant, and tool agents to synthesize compositional trajectories [Chen et al., 2025a]. TOUCAN scales this direction to 1.5M trajectories grounded in real-world MCP servers and validates execution success, tool coverage, ordering, completeness, and conciseness [Xu et al., 2025f]. FunReason-MT constructs difficult goal-directed traces from environment-API graphs with dependency constraints and iterative failure feedback [Xu et al., 2025d]. UniToolCall is a hybrid resource: it unifies real and synthetic examples under a Query–Action–Observation–Answer representation and evaluates them with dynamic sandbox execution [Liang et al., 2026].

Environment synthesis provides another source of agent data. WebWorld learns a generative web simulator from large-scale open-web interactions and uses it to produce long-horizon training trajectories [Xiao et al., 2026]. ClawGym combines benchmark construction with synthetic persona-driven tasks, realistic mock workspaces, sandboxed trajectory collection, hybrid verification, SFT, and RL [Bai et al., 2026]. Agent-World discovers executable tool ecosystems and generates controllable-difficulty tasks according to capability gaps during training [Dong et al., 2026]. Nex-N1 integrates automatic agent and environment construction with grounded trajectory generation and quality filtering [Nex-AGI Team, 2025], while ClawEnvKit automatically converts natural-language requirements into task specifications, tools, fixtures, scoring configurations, and isolated evaluation environments [Li et al., 2026b]. Data synthesis is also becoming increasingly model-aware. Agent-Frontier uses tool-augmented knowledge expansion and ZPD calibration to retain tasks that the base model cannot solve unaided but an agent can solve with tools, followed by verification, deduplication, and human-review routing [Chen et al., 2025b].

2.3. Summary: Substrate Dynamics, Epistemic Traps, and Selection Trade-offs

2.3.1. The Scale-Density Frontier: From Brute-Force to Curatorial Selection

A cross-examination of existing substrates reveals that the foundational paradigm of LLM training has decoupled from pure scale maximization, pivoting toward **epistemic density** and **distributional alignment**. The critical transition between pretraining, SFT, and RL represents a narrowing of the data-model boundary, where raw token volume is sacrificed for behavioral control and reasoning fidelity.

- **Pretraining Substrates (Token-Driven vs. Filter-Driven):** Early foundation training defaulted to massive web-crawl dumps. Studies such as DCLM and FineWeb suggest that aggressive quality filtering can substantially reduce the amount of training data while maintaining or improving downstream performance under specific training settings. The core trade-off lies in **compute allocation**: shifting budgets from optimization scaling (more steps over raw text) to curatorial preprocessing (filtering noisy tokens long before gradient descent).
- **SFT Substrates (The Surface Alignment Hypothesis):** Synthesizing findings from LIMA, FLAN, and the Tulu mix reveals a fundamental operational boundary: SFT datasets often contribute

strongly to instruction-following and response style, while their contribution to knowledge acquisition depends on the task, data quality, and training setting. Brute-force scaling of SFT instruction pairs beyond a certain threshold may yield diminishing returns and increase the risk of **over-fitting to linguistic control styles**, while low-volume, high-density reasoning sequences (e.g., LIMO, s1) are highly effective at activating the latent world knowledge embedded during pretraining.

2.3.2. Epistemic Traps and Failure Modes Across Modalities

When selecting or constructing a data substrate, practitioners routinely fall into predictable "epistemic traps." We synthesize the most acute failure modes documented in the literature across the three primary application areas:

- **The Over-Refusal Syndrome in Safety Substrates:** Adversarial and safety datasets (e.g., HarmBench, XSTest) are heavily weighted toward negative contrastive tokens (harmful queries paired with standard refusal templates). An evidential review shows that over-indexing on uniform refusal substrates corrupts the model's semantic boundary, triggering **exaggerated safety behaviors** (over-refusal) where benign prompts containing sensitive trigger words are falsely flagged.
- **The Mimicry Trap in Knowledge Distillation:** Substrates distilled entirely from proprietary teacher models (e.g., WizardCoder, UltraFeedback) generate misleading evaluation metrics. Because student models replicate the superficial stylistic markers, confidence calibration, and formatting fluency of the teacher, they create an **illusion of competence** while their underlying factual accuracy and out-of-distribution reasoning may remain weaker than their surface-level fluency suggests.
- **Static Bias in Agent Trajectories:** Early tool-use datasets (e.g., API-Bank, ToolLLM) rely on static, human-curated API log history. This can create an **exposure-bias-like** failure mode: when deployed online, the model drifts from the static data distribution due to subtle environment changes, leading to cascading errors that it cannot self-correct, because the training substrate completely lacked negative or failure-recovery trajectory logs.

2.3.3. From Static Artifacts to Executable Data-Producing Environments

Recent agentic datasets expose a change in the form of the data substrate, rather than merely the addition of another application domain. Traditional corpora and trajectory datasets are stored artifacts: once collected, their task distribution and interaction paths are largely fixed. In contrast, executable environments such as sandboxed software systems, web simulators, and generated tool ecosystems can produce new tasks, actions, observations, failures, and recovery trajectories on demand.

This distinction matters for data-centric analysis because the environment itself becomes part of the data infrastructure. Static logs remain valuable for reproducibility and inexpensive offline training, but they can become stale as tools, interfaces, or task distributions change. Executable environments can instead support controlled perturbation, repeated interaction, and trajectory generation targeted to current capability gaps, although they introduce substantial verification and computational costs. We therefore view the recent transition from stored trajectories toward executable environments as an expansion of the data substrate from data artifacts to data-producing systems.

2.3.4. Evidence-Informed Decision Matrix

To transition this survey from a structured bibliography to an actionable engineering guide, Table 11 formalizes an evidence-weighted decision framework for selecting initial raw data substrates based on the engineering target and resource constraints. The recommendations in this table summarize recurring evidence and engineering considerations. They are setting-dependent rather than universal prescriptions.

Table 11 | Evidence-Informed Decision Matrix across LLM Lifecycle Phases

Substrate Archetype	Representative Baselines	Core Strength (Pros)	Primary Vulnerability (Cons)	Optimal Lifecycle Phase & Mitigation Strategy
Massive Open-Web	Dolma, GneissWeb, SlimPajama	Unparalleled world knowledge coverage and long-tail domain breadth.	Extreme semantic noise, PII leakage, high copyright/licensing legal risks.	**Pretraining (Initial Base Phase).** Requires aggressive proxy-model deduplication and quality scoring (e.g., DCLM) to prune low-value documents.
Teacher-Distilled	WizardLM, UltraFeedback, Magpie	Rapid control style alignment; low direct human annotation overhead; complex format compliance.	Encourages superficial style mimicry; inherits hidden hallucinations and systemic biases of the teacher.	**SFT (Cold-Start Stage).** Often useful for SFT or cold-start stages when paired with correctness-based filters and diverse prompts; the effective volume is task- and model-dependent.
Executable Env.	ClawGym, Multi-SWE-bench, TOUCAN	Step-level verifiability; reduced dependence on static logs; includes realistic environment error-recovery paths.	Hyper-narrow domain coverage (restricted to code/math-/digital workflows); computationally expensive sandbox scaling.	**Post-Training / RL Scaling (System 2 Induction).** Useful for post-training and RL when actions and outcomes can be reliably executed or checked. Evaluate transfer across task templates, tool versions, and environment variants.

3. Data Creation and Selection: Content and Composition Transformations

Once the raw data substrate has been gathered, the data lifecycle transitions from resource accumulation to data construction, transformation, and selection. We define Data Creation and Data Selection as methods that modify sample content, format, labels, or inclusion decisions before the data are consumed by the model's optimization process. To anchor these spatial transformations in state-of-the-art engineering practices, we observe a clear paradigm divergence between our anchor lineages. LLaMA and DeepSeek lineages illustrate different engineering emphases rather than a strict paradigm divergence. Conversely, the DeepSeek lineage pioneers autonomous synthesis (Creation), using micro-annotation for cold-start bootstrapping (as seen in DeepSeek-R1) coupled with massive execution-guided self-play and symbolic generators to synthesize multi-step reasoning traces.

3.1. Data Creation

3.1.1. Annotation

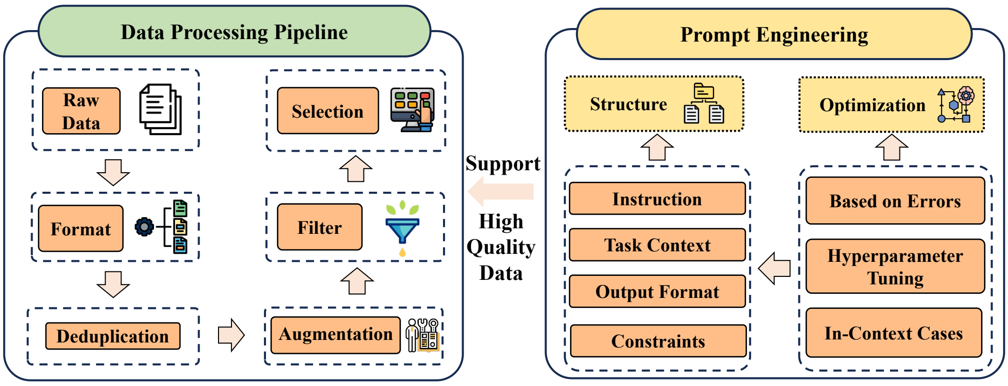

Figure 4 | The Annotation process of LLMs. The focus is more on using prompt engineering to generate a large volume of initial data.

Annotation is the first step in embedding human knowledge into training corpora. Rather than merely labelling raw text, we treat it as the process of converting implicit expertise into explicit, model-readable signals. This subsection first reviews data-processing operations, formatting, deduplication, augmentation and rule-based selection, that turn large-scale web dumps into annotation-ready material. We then summarise prompt-engineering techniques that guide LLMs to produce consistent pseudo-labels or structured outputs, reducing the need for exhaustive manual effort while preserving quality. We overview key techniques in Figure 4.

Data Processing Data Processing supports Annotation by refining large-scale unsupervised data into cognitive-ready inputs. These operations ensure data meets the quality and scale demands of data-centric LLM development, laying a stable foundation for subsequent cognitive engineering. These include formatting (unifying data structures for LLM compatibility) [Soldaini et al., 2024], deduplication (removing redundant entries) [Shen et al., 2025a], augmentation (generating diverse data variants) [Nguyen et al., 2025], and selection (Wang et al. [2025b], Token Cleaning [Pang et al., 2025], AutoDCWorkflow [Li et al., 2025d]) (retaining task-relevant, high-quality data). These steps ensure processed data meets the scale and quality demands of Data-Centric LLM development. Specifically, Soldaini et al. [2024] releases Dolma (a 3-trillion-token English corpus with diverse sources), documents it, shares data curation insights via its intermediate state analyses, and opens the curation toolkit. The DCLM [Li et al., 2025a] benchmark enables experimentation with data curation strategies (e.g., deduplication, filtering, and data mixing) across model scales ranging from 412M to 7B parameters. AutoClean [Shen et al., 2025a] explores the potential of the LLM agents in automating data cleaning methodologies. Recycling the Web [Nguyen et al., 2025] explores ways to transform and recycle data discarded in existing filtering processes. Wang et al. [2025b] introduces an efficient verification strategy, which facilitates the rapid evaluation of the impact of the data on LLM training with minimal computational cost, optimizes the selection of positive and negative samples, and proposes an efficient data filtering pipeline. Token Cleaning [Pang et al., 2025] investigates token quality from a noisy-label perspective and proposes a generic token cleaning pipeline for SFT tasks. AutoDCWorkflow [Li et al., 2025d] is an LLM-based pipeline to generate data cleaning workflows automatically. In resource-constrained scenarios where computational budget is limited, rule-based data processing (e.g., heuristic filtering) remains the most cost-effective baseline; however, for downstream tasks requiring high semantic nuance, such as complex instruction following, it should be complemented by model-based annotation to prevent the loss of tail-knowledge often discarded by rigid rules.

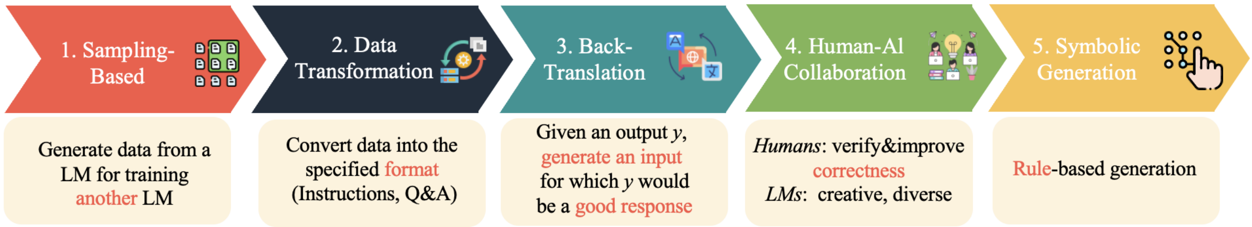

Figure 5 | Synthesis methods. We organize the synthesis workflow into five representative stages, from raw inputs to structured or logically grounded outputs.

Prompt Engineering While rule-based processing cleans the raw signal, it cannot generate new supervision. To bridge the gap between unlabeled corpora and supervised requirements, we turn to prompt engineering as a mechanism for semi-automated label generation.

1) *prompt **structure** design* Structured prompt design forms the foundation for consistency. By defining standardized frameworks (e.g., Instruction + Task Context + Output Format + Constraints), ambiguity in LLM-driven annotation can be reduced. Recent works propose systematic Prompt Pattern Catalogs that summarize reusable prompting templates [White et al., 2023], as well as comprehensive surveys that systematize prompt structure by establishing vocabularies and taxonomies of prompting patterns [Schulhoff et al., 2025], providing clearer frameworks for consistent design across domains. Chain-of-Table [Wang et al., 2024f] incorporates tabular data into reasoning chains for effective labeling of structured datasets. Iterative refinement approaches, such as Self-Refine [Madaan et al., 2023], leverage model self-feedback to reduce noise and enhance annotation quality without external supervision. Similarly, code-based prompting [Puerto et al., 2024] has been shown to transform unstructured tasks into executable forms, improving annotation consistency for logical or rule-based labeling. Other techniques, including demonstration ensembling [Khalifa et al., 2023] and task decomposition [Wang and Zhao, 2024], further reduce ambiguity by either aggregating multiple perspectives or breaking down complex tasks into simpler subtasks, thereby improving annotation robustness.

2) *prompt **optimization** strategies* Beyond design and techniques, optimization strategies aim to make annotation more adaptive and scalable. Meta-prompting methods, such as PE2 [Ye et al., 2024], guide LLMs to refine prompts with structured templates, while logic-driven frameworks like LoT [Zhao et al., 2024d]. Further, optimization of in-context exemplars, through controlling their number, order, and relevance, has been demonstrated to systematically improve annotation stability [Brown et al., 2020, Lu et al., 2022]. Constraint-driven prompting explicitly embeds annotation guidelines into the prompt design, helping enforce consistency and reducing redundancy [Lester et al., 2021]. Recently, automated prompt optimization frameworks (e.g., OPRO [Yang et al., 2024b]) have shown the potential to let LLMs themselves iteratively refine prompts, thereby minimizing annotation variance and enhancing data quality.

3.1.2. Synthesis

When human annotations are scarce or expensive, targeted generation becomes essential. Synthesis here refers to any automated procedure that creates new training examples rather than simply relabelling existing ones. This section highlights cutting-edge generation approaches that do not merely expand data volume, but enrich the epistemic diversity of training data to fill cognitive gaps. Figure 5 outlines key methods.

Table 12 | Comparison of Sampling-Based Methods. Based on model access permissions and data generation paradigms, sampling methods are classified into four subcategories. “Subcategory” column includes: (1) KD White-Box, (2) KD Black-Box, (3) Seed-Based, and (4) Zero-Generation, and “Key Mechanism” column refers to the dimension of acquiring knowledge from the teacher model.

Method	Year	Key Mechanism	Subcategory				Domain
			1	2	3	4	
Sequence-Level KD	2016	Answer		✓			General Domain
Self-Instruct	2023	Question+Answer			✓		General Domain
Distilling Step-by-Step, Fine-tune-CoT	2023	Question+CoT		✓			CoT Reasoning
Unnatural	2023	Process/Distribution				✓	General Domain
MiniLLM	2024	Logits	✓				General Domain
Synthetic Data (Almost) from Scratch	2024	Question+CoT				✓	Vertical Domain
WizardLM	2024	Question+Answer			✓		General Domain
Bonito	2024	Answer			✓		General Domain
Magpie	2025	Answer				✓	General Domain
AQuilt	2025	Question+Answer				✓	Vertical Domain
Absolute Zero, Synthetic Data RL	2025	RL Reward				✓	RL
Condor	2025	Process/Distribution				✓	Vertical Domain

Sampling-Based Sampling-based generation extracts or generates data samples from a distribution to expand training datasets, with a focus on encoding cognitive patterns (e.g., reasoning steps, task logic) rather than just increasing size. It serves as the initial data acquirer in the LLM data lifecycle. It addresses the “data availability” issue, providing raw materials for all subsequent processing. Without sampling, downstream methods lack targeted data. For specific comparisons, please refer to Table 12. When the goal is to bootstrap a model from scratch with minimal human data, Seed-Based generation is often attractive. Conversely, when a strong teacher model is available, Knowledge Distillation offers superior density and coherence, though it carries the risk of inheriting the teacher’s biases.

Knowledge Distillation Knowledge distillation comprises two components: 1) *White-Box Distillation*, where the internal parameters of the model are known to derive distribution data. Common methods include MiniLLM [Gu et al., 2024b], GKD [Agarwal et al., 2024] and OKD [Rao et al., 2024b], all of which combine student-driven exploration with teacher-provided distribution information under different exploration scenarios to distill knowledge from large models. MiniLLM [Gu et al., 2024b] first replaced the forward Kullback-Leibler divergence (KLD) objective in the standard KD approaches with reverse KLD. Instead of solely relying on a fixed set of output sequences (Vanilla KD), GKD [Agarwal et al., 2024] trains the student on its self-generated output sequences by leveraging the teacher’s feedback on these sequences. 2) *Black-Box Distillation*, where the model’s parameters are unknown. This approach distills the teacher’s knowledge by utilizing the model’s sequence-level output: the entire sentence (Sequence-level KD [Kim and Rush, 2016]), rather than the distribution of each individual token. Through this method, the model employs various techniques to distill its reasoning steps, processes [Jung et al., 2024] and dataset distribution [Wu et al., 2025, Yang et al., 2024c]. Sequence-level KD [Kim and Rush, 2016] has been applied extensively to current LLM distillation, specifically distilling entire sentences from LLMs rather than individual tokens. Distilling Step-by-Step [Hsieh et al., 2023] extracted LLM rationales as additional supervision to train small models within a multi-task framework. Fine-tune-CoT [Ho et al., 2023] retained the core of Sequence-level

KD while emphasizing the distillation of the CoT process.

From Scratch This approach generates data without heavy reliance on existing datasets, bootstrapping cognitive signals from minimal inputs: 1) *Seed-based Generation* This type of method assumes access to a limited number of seed tasks or examples and iteratively expands them. Self-Instruct [Wang et al., 2023d] demonstrated that only a few hundred seed instructions are sufficient to bootstrap a much larger task pool via iterative self-generation. Building on this idea, WizardLM [Xu et al., 2023a] proposed the Evol-Instruct strategy, which progressively transforms simple instructions into more complex ones by adding constraints, refining concepts, and increasing reasoning steps. Other extensions exploit synthetic dialog scenarios [Finch and Choi, 2024] to further diversify the generated data. 2) *Zero Generation* A growing line of work removes even this dependency, aiming at zero-human-data generation. Absolute Zero [Zhao et al., 2025a] trains reasoning models through self-play reinforcement learning: the same model acts as both problem proposer and solver, while an external executor verifies correctness to provide stable reward signals. Similarly, Synthetic Data RL [Guo et al., 2025b] shows that a task definition alone, describing inputs, outputs, and goals, can be sufficient; an instructor LLM generates question-answer pairs based on retrieved external knowledge. Magpie [Xu et al., 2025e] leverages the implicit instruction distribution learned by aligned LLMs, prompting them with minimal pre-query templates to automatically produce high-quality and diverse user instructions without seeds. Knowledge-driven approaches such as Synthetic Data (Almost) from Scratch [Li et al., 2024b] and Condor [Maosongcao et al., 2025] instead construct taxonomies or knowledge trees to ensure broad domain coverage, difficulty control, and balanced diversity. AQuilt [Ke et al., 2025] constructs instruction-tuning data for any specialized domains from corresponding unlabeled data, including Answer, Question, Unlabeled data, Inspection, Logic, and Task type.

Data Transformation It modifies or augments existing datasets to enhance their cognitive value, reusing available data to build new reasoning patterns. It standardizes raw data and balances data distribution to fit LLM input requirements. It refines sampling outputs into "processable data", laying the foundation for Back-Translation and Human-AI Collaboration, avoiding format-induced inefficiencies in later methods. *Knowledge Graph* The transformation of existing data, modifying or augmenting existing datasets through Knowledge Graphs (wiki or custom graphs for different domains), can be an effective method to produce diverse dialogues. SODA [Kim et al., 2023] contextualizes social commonsense knowledge from a knowledge graph to distill an exceptionally broad spectrum of social interactions. Condor [Maosongcao et al., 2025] incorporates World Knowledge Trees and Self-Reflection Refinement to produce high-quality SFT data at scale. Tree-KG [Niu et al., 2025] focuses on the integration of structured domain text and semantic technologies. *Rephrasing Documents* For more disorganized web text, we can create new data by paraphrasing [Maini et al., 2024]. Given the massive volume of data at this stage, it can be divided into manageable chunks for easy scaling to obtain sufficient data. MAmmoTH2 [Yue et al., 2024b] naturally transforms existing instruction data from the pre-training web corpus. Some projects also incorporate personas [Ge et al., 2025] at this stage to further expand the scope. When addressing a specific task, the data preparation process (Retrieval-Augmented Generation, RAG) typically involves two core steps: first, retrieving relevant datasets or documents that align with the task's requirements [Gandhi et al., 2024, Ziegler et al., 2025]; second, transforming the retrieved materials into data formatted to meet the desired standards of the target task (Reformatted [Fan et al., 2024]).

Back-Translation Back-Translation is a low-cost semantic enhancer using "source-target-source" translation (e.g., Chinese-English-Chinese) to generate paraphrased data while preserving meaning. It leverages Data Transformation's standardized outputs to minimize translation bias, expanding high-quality data scales. It often collaborates with Human-AI: its outputs are later calibrated for accuracy. For specific comparisons, please refer to Table 13. It has evolved from machine translation to LLM alignment:

Table 13 | Comparison of Back-Translation Methods for Large Language Models. End-to-End Pipeline: ✓indicates a complete automatic translation loop, × indicates partial or manually guided processing.

Method	Year	Generation Strategy	Quality Control	End-to-End Pipeline
Self-Alignment with Instruction Backtranslation	2024	Back-translation for self-alignment data synthesis	Rule-based filtering	×
Instruction Back-and-Forth Translation	2024	Forward/back translation with LLM rewriting	Manual validation + filtering	×
LongForm	2024	Reverse-instruction generation from documents	Human sampling	×
Safer-Instruct	2024	Reverse instruction + preference construction for safety alignment	Expert model evaluation	×
Cycle-Instruct	2025	Dual self-training with cycle consistency (seed-free)	Cycle consistency check	✓
REER	2025	Reverse-engineered reasoning for open-ended generation	Human + model verification	✓

Machine Translation Back-translation has long been a cornerstone in machine translation, providing synthetic parallel data to enhance model performance. Recently, this paradigm has been extended to instruction-tuned LLMs. Self-Alignment with Instruction Backtranslation [Li et al., 2024f] employs synthetic instructions generated through back-translation to expand alignment datasets, yielding substantial improvements in instruction-following. Building on this, instruction back-and-forth translation [Nguyen et al., 2024] further enhances data quality by not only backtranslating web corpus texts but also rewriting responses via LLMs, resulting in cleaner and higher-quality datasets.

Alignment Early work such as LongForm [Köksal et al., 2024] introduces reverse instructions, where LLMs generate instructions from human-written documents, thus providing a cheaper and more natural instruction-tuning corpus. Building on this, Safer-Instruct [Shi et al., 2024a] leverages reverse instruction tuning and expert model evaluation to automatically construct preference data sets, particularly improving alignment in safety-sensitive domains. More recently, Cycle-Instruct [Shen et al., 2025b] has advanced this line of research by employing dual self-training and cycle consistency, allowing instruction tuning without any human-provided seeds, thus pushing back-translation methods toward fully autonomous data generation.

Human-AI Collaboration However, purely automated synthesis often lacks the nuance of human intent. Thus, incorporating human-in-the-loop mechanisms becomes essential to calibrate the quality of machine-generated data. It functions as the quality calibrator across mid-to-late stages. It combines AI's scalability (e.g., self-refinement) with human expertise (e.g., error correction) to validate outputs from Sampling, Transformation, and Back-Translation. It ensures data reliability, guides Symbolic Generation's rule design, and finalizes high-quality data for LLMs.

Self-Generation It uses "self-driven" mechanisms (self-alignment, self-refinement, self-play) to enhance model cognition: 1) *Response Refinement* Among these types of papers [Sun et al., 2024], SELF-ALIGN [Sun et al., 2023] guides LLMs to self-align with human behavior using a small set of human-defined principles, minimizing human supervision. Self-refine [Ranaldi and Freitas, 2024] enables LLMs to optimize outputs via self-generated feedback without extra training data iteratively. SPIN [Chen et al., 2024b] boosts LLM capabilities through self-play fine-tuning. 2) *Multi-Agent and Reflective Generation* Self-critiquing [Saunders et al., 2022] allows models to self-evaluate outputs, identify flaws, and correct errors to improve quality, while AutoIF [Dong et al., 2025a] uses self-play loops, combining self-evaluation and adjustment to drive iterative model improvement. Beyond single-model synthesis, recent methods introduce multi-agent debate and reflection mechanisms to improve data quality. Debate & Reflect [Zhou et al., 2025b] employs interactions between strong teacher models and smaller student models, recording arguments in a structured graph and distilling them into preference trees via tree-structured direct preference optimization (T-DPO). This allows student models to learn which answers are correct and why certain reasoning paths are preferable. Similarly, SAND-Math [Manem et al., 2025] applies self-play and difficulty calibration to generate challenging mathematical problems. These approaches highlight a shift from passive data generation to interactive and self-correcting synthesis.

Error-Guided This method targets model weaknesses by generating data that addresses cognitive gaps: 1) In domain-specific weakness detection [Wang et al., 2023a, Zhao et al., 2024b], key approaches include AutoDetect, using a multi-agent framework [Cheng et al., 2024b] for iterative detection via challenging tasks, APT [Rao et al., 2025b], a weakness-driven iterative preference learning framework, and SwS [Liang et al., 2025] an RL-based synthesis of targeted problems from repeated failure samples to address weaknesses. 2) In math-specific error improvement, recent work includes: Li et al. [2024g] focused on evaluating LLMs' mathematical reasoning from a reviewer's perspective, designing four fine-grained error-related tasks (error existence detection, first error step localization, error type classification, and error correction). Setlur et al. [2024] used reinforcement learning on incorrect synthetic data via advantage-weighted rewards, boosting efficiency equivalent to 8× more positive samples. Self-Error Instruct [Yu et al., 2025] grouped bad cases by error type to generate generalized training data for better correction. Lemma [Pan et al., 2025] is a framework using teacher models to create errors and reflection-correction data with two training strategies for stronger self-correction. RISE [Xu et al., 2025c] injects predefined subtle errors into pivotal tokens during reasoning to construct hard pairs for error mitigation.

Table 14 | Comparison of Symbolic Generation Methods. Verifiability Level (Verif.) is categorized by the paper's self-assessment as: High (formal verification/executable verifiability), Medium (rules or templates + manual sampling). Multilingual Support column (Multi-Lang.): ✓indicates the paper explicitly provides non-English experiments or claims support, × indicates English only.

Method	Year	Core Idea	Verif.	Synthesis Form / Intermediate Repr.	Multi-Lang.
Logic-LM	2023	Hybrid solver+LLM: translate NL logic problems into modules and call external symbolic solver	High	Problem Formulator/Symbolic Reasoner	×
NSR	2024	Neuro-symbolic reasoning: LLM→symbolic facts→symbolic engine→NL	High	Symbolic Facts	×
MURI	2024	Reverse instructions + symbolic templates for low-resource languages	Medium	Templates + back-trans	✓
CSDG	2024	Context-sensitive declarative grammar; executor judges truth	High	Grammar productions	✓
SR-LCL	2024	Learnable concept library for symbolic regression	High	Basis-Function Expressions	×
SymbCoT	2024	Chain-of-thought with symbolic structures; predicates/rules verified on-the-fly	High	Predicate Logic Steps	×
ProtoReasoning	2025	Prototype-driven: induce logic prototypes, instantiate new chains	Medium	Prototype Instances	×
Aristotle	2025	Logic-complete decompose-search-resolve pipeline	High	Integrated Dualpath Reasoning Process	✓
Com2	2025	Causal-guided commonsense: causal Graph→counterfactuals	Medium	Causal graph	×
SC-NeuroSymbolic	2025	a Bilateral factuality evaluation function; LLM-grounded interpretation	High	Formal Semantics for a Paraconsistent Logic	×
SymbolicThought	2025	Relation reasoning: first-order logic on social relations + theorem prover	High	Editable Character Relationship Graphs	×
AdaCoT	2025	Adaptive multilingual CoT: language-agnostic symbolic Intermediate Representation	Medium	Language-agnostic Intermediate Representation	✓
NSAR	2025	Merges symbolic reasoning with neural inference	High	Shared symbolic facts	✓

Symbolic Generation As shown in Table 14, to mitigate the inherent hallucination risks of statistical text generation, Symbolic Generation injects formal logic into the synthesis pipeline. This approach constructs multi-modal (text + symbol) data pools that ground LLM reasoning in verifiable structures rather than mere linguistic probability. For domains intolerant to hallucination, such as mathematics or code, Symbolic Generation is preferable to pure text-based synthesis, as it introduces an external verifier (e.g., a code compiler or logic solver) to guarantee the correctness of the training signal.

Neuro-Symbolic Integration & Solvers: This stream enhances logical consistency by coupling LLMs with formal systems. Early hybrid frameworks like Logic-LM [Pan et al., 2023] offload rigorous deduction to external symbolic solvers to ensure faithfulness. Moving toward intrinsic integration, NSR [Fang et al., 2024] embeds symbolic reasoning directly into neural architectures, while SR-LCL [Grayeli et al., 2024] employs learned concept libraries for symbolic regression. To refine intermediate reasoning, SymbCoT [Xu et al., 2024] and ProtoReasoning [He et al., 2025a] restructure CoT prompting with symbolic predicates and prototype logic, enabling step-by-step verification of the generation process. *Logic-Complete Pipelines & Interpretability*: Recent work focuses on end-to-end reliability and causal grounding. Aristotle [Xu et al., 2025b] introduces a “decompose-search-resolve" pipeline to formalize complex problem-solving, while Com^2 [Xiong et al., 2025] synthesizes data specifically for causal commonsense reasoning via counterfactual graphs. Emphasizing interpretability, SC-NeuroSymbolic [Allen et al., 2025] and Symbolic Thought [Zhao et al., 2025c] utilize formal semantics and relationship graphs to enforce consistency in social and logical reasoning tasks. *Multi-lingual Symbolic Extension*: Symbolic logic serves as a language-agnostic bridge to scale high-quality data for non-English domains. MURI [Köksal et al., 2024] leverages reverse-instruction templates to generate instruction-tuning data for low-resource languages. To ensure grammatical and factual precision across languages, Sileo [Sileo, 2024] utilizes context-sensitive declarative grammars, while AdaCoT[Zheng et al., 2025b] adapts CoT reasoning for cross-lingual tasks. Additionally, Nezhad and Agrawal [Nezhad and Agrawal, 2025] propose extracting explicit symbolic facts to maintain reasoning consistency in multilingual neuro-symbolic systems.

3.2. Data Selection

Table 15 | Comparison of Data Selection Methods for LLMs.

Taxonomy	Stage	Methods	Year	Key Mechanism
Surface-Level Heuristics	SFT	Impossible Distillation	2024	Rule-based Similarity
	SFT	DiverseEvol	2023	Model-based Similarity
	SFT	QDIT, DEITA	2024	Model-based Similarity
	SFT	Instag	2023	Semantic Tags
Loss Gradients	SFT	LESS	2024	Influence Formulation
	Pretrain	MATES	2024	Proxy Model
	SFT	G-Vendi	2025	Proxy Model
Reward Models	SFT/RL	Alpagasus (SFT), Prometheus (RL)	2024	Model Judgment
	SFT	LIFT	2023	Data Distribution
Metric	SFT	LTD	2023	Cluster
	SFT/Pretrain	GREATS	2024	Utility Optimization
	SFT	IFD	2024	Instruction-Following Difficulty Score
Composite Strategy	SFT	LIMA	2023	Human Selection
	SFT	MoDS	2023	Quality, Coverage and Necessity
	SFT	SelectIT	2024	Model Self-Assessment
	Pretrain	Dataman	2025	Multiple Defined Dimensions

Data Selection is a critical step in the LLMs pipeline: it ensures only high-fidelity data, aligned with task cognitive requirements, shapes model intelligence. High-quality data enhances downstream task performance (extrinsic effects), but evaluating all data points is impractical. Thus, intrinsic metrics (data characteristics like diversity and quality) are used to assess cognitive value. Figure 6 outlines evaluation dimensions. Table 15 presents their comparative dimensions.

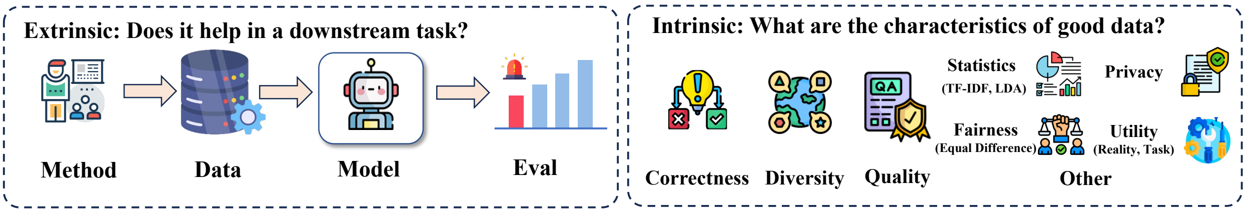

Figure 6 | Evaluation of synthetic data.

3.2.1. Diversity

Diversity emerges as a key complementary strategy to improve data quality and avoid redundancy. In the pre-training phase, prioritizing Diversity (e.g., via gradient-based selection) is critical to prevent mode collapse and ensure broad generalization. However, during the SFT stage, this priority must shift towards Quality, as noisy data in the fine-tuning phase causes disproportionate degradation in model alignment. *1) Surface-Level Heuristics* This selection relies on surface-level heuristics and employs multiple practical metrics to identify and remove similar examples: methods like Impossible Distillation [Jung et al., 2024] use Rouge-L to define and filter overlapping samples; QDIT [Bukharin et al., 2024], and DEITA [Liu et al., 2024d] leverage embedding similarity to eliminate redundant data; and Instag [Lu et al., 2024a] adopts semantic tags to distinguish and retain semantically diverse examples, all aiming to ensure the selected training data covers richer subspaces and better supports efficient training. *2) Loss Gradients* The gradient information of the model can be used for data selection. The higher data diversity means more robust models. LESS [Xia et al., 2024a] selects influential data for targeted instruction tuning through low-rank gradient similarity search. MATES [Yu et al., 2024b] used a small data influence model that continuously adapts to the changing data preferences of the pretraining model and selects training data to optimize the efficacy of the pretraining process. G-Vendi [Jung et al., 2025] quantifies diversity via the entropy of model-induced gradients.

3.2.2. Quality

Quality selection removes low-quality, noisy, or irrelevant data to ensure that LLMs are trained on reliable and useful information. While Reward Models provide fine-grained quality signals, they are computationally expensive for large-scale filtering. In scenarios requiring rapid iteration, Metric-based approaches offer a more scalable, albeit coarser, alternative for pruning low-value samples. *1) Reward Models* Generally speaking, a scoring model can be used to judge the quality of data. There are two approaches for this: one is the direct discriminative method like Prometheus [Kim et al., 2024], which judges the relative quality of A and B; the other is the generative method like Alpagasus [Chen et al., 2024a], which directly generates a score for a single piece of data. There are also works (LIFT [Xu et al., 2023c]) that do not utilize the output of LLMs. They implicitly broaden the data distribution to cover more high-quality subspaces, eliminate redundancy, and focus on high-quality segments in the overall data subspaces. *2) Metric* Different artificially designed metrics can be used to select appropriate training data. Chen et al. [2023a] proposed a coreset-based and task-related data selection method named LTD with embeddings. GREATS [Wang et al., 2024a] formulates batch selection as an optimization problem and employs greedy algorithms and Taylor expansions to solve the optimization problem. IFD [Li et al., 2024c] hypothesize that LLMs can first be trained on a carefully curated subset of instruction data to acquire basic instruction-following capabilities. This foundational skill then enables the model to better estimate instruction difficulty and overall data quality. Deepseek R1 [Guo et al., 2025a] demonstrated that using correctness (final answer verification) [Zelikman et al., 2022] as a reward significantly enhances the mathematical capabilities.

3.2.3. Composite Strategy

Recognizing that neither diversity nor quality alone guarantees optimal performance, recent research has moved towards composite strategies that optimize these dimensions simultaneously. LIMA [Zhou et al., 2023] demonstrated early on that a small number of carefully prepared samples could trigger the model's capabilities. MoDS [Du et al., 2023] focused on instruction selection through three criteria: quality (instructional data fidelity), coverage (variety of instruction types), and necessity (impact of instruction on LLM fine-tuning). SelectIT [Liu et al., 2024b] leveraged the foundational capabilities of LLMs themselves, particularly their inherent uncertainty, to effectively select high-quality IT data. Dataman [Peng et al., 2025] employed multiple dimensions to assign diverse scores to data.

3.3. Summary: Creation and Selection Trade-offs

3.3.1. The Knowledge Distillation Spectrum: Fidelity vs. Parameter Exposure

A critical synthesis of current language model distillation literature reveals a fundamental trade-off between semantic exploration autonomy and optimization stability. When transforming raw substrates via teacher models, the choice between white-box and black-box distillation is governed by parameter accessibility and compute budgets:

- **White-Box Gradient Alignment:** Frameworks utilizing internal parameter distributions (e.g., *MiniLLM*, **GKD**) modify the optimization objective by substituting standard forward Kullback-Leibler divergence (KLD) with reverse-KLD. This encourages the student to place less probability mass on low-probability teacher modes and has been reported to mitigate some token-level errors and vocabulary drift in specific settings. However, this strategy is strictly bounded by white-box access, making it inapplicable to proprietary commercial foundations.
- **Black-Box Rationale Distillation:** Conversely, black-box approaches (e.g., *Distilling Step-by-Step*, *Fine-tune-CoT*) leverage sequence-level text API outputs . While bypassing parameter constraints, black-box distillation may suffers from **exposure bias** and mimicry traps. The student replicates the stylistic confidence and formatting syntax of the teacher without internalizing the underlying logical constraints, which may lead to weaker out-of-distribution generalization.

3.3.2. The Autonomy Frontier and the Evolving Role of Human Curation

As synthetic-data pipelines move from seed-based mutation toward self-play, iterative generation, and increasingly automated data flywheels, the amount of direct per-example human annotation can decrease substantially. The reviewed literature, however, does not imply that human involvement simply disappears. Instead, automation changes where human knowledge enters the lifecycle.

In less automated pipelines, human effort is concentrated on writing or labeling individual examples. In more autonomous pipelines, it increasingly appears in task specification, annotation guidelines, verifier construction, admissible-action boundaries, adversarial test design, and the validation of exceptional or ambiguous cases. Self-play and multi-agent generation can expand coverage and expose model-specific weaknesses, but their outputs remain sensitive to the objectives, verification rules, and seed distributions supplied to the loop. We therefore interpret the current trend as a shift from **instance-level supervision toward system-level specification and verification**, rather than a monotonic removal of human curation.

3.3.3. Symbolic Grounding vs. Statistical Probability

The stochastic nature of autoregressive tokens fundamentally constrains pure text-based synthesis . For reasoning-intensive domains intolerant to hallucination (e.g., STEM, software compilation), **Symbolic Generation can be particularly useful when reliable external solvers, compilers, or verifiers are available.** By embedding external solvers, verification graphs, or code compilers (e.g., *Logic-LM*, *NSR*, *Symbolic Thought*) into the creation pipeline, the data morphology shifts from ungrounded token strings to verifiable neuro-symbolic pairs. The external verifier introduces a hard correctness constraint that acts as an offline reward signal, helping detect some incorrect samples before they are used for training.

3.3.4. From Universal Data Quality to Model-Conditional Utility

Data-selection methods are commonly organized around intrinsic properties such as quality, diversity, difficulty, or redundancy. Taken together, however, recent gradient-, influence-, uncertainty-, and capability-aware methods suggest a broader distinction between data quality as an intrinsic property and **data utility as a relation between a sample and the current model**.

Surface and embedding heuristics remain attractive at web scale because they can remove obvious noise or redundancy without repeatedly querying the target model. Their limitation is that a globally high-quality sample may provide little additional learning signal to a model that already masters it. Model-aware approaches instead estimate utility through losses, gradients, uncertainty, influence, or observed errors, allowing selection to shift as model capabilities change. This additional adaptivity comes at a higher computational cost and can over-concentrate training on noisy or transient failures. The resulting trade-off suggests that static and model-aware selection are better viewed as complementary regimes rather than competing definitions of a universally optimal dataset.

3.3.5. Evidence-Informed Decision Matrix

To guide engineering practices under varying resource ceilings and data availability constraints, Table 16 distills the application boundaries, failure modes, and mitigation strategies of the core creation and selection methodologies. These recommendations are conditional on teacher quality, verifier reliability, proxy–target compatibility, model scale, and available compute.

4. Data Ingestion Strategies: Temporal Scheduling and Runtime Information Use

Following data creation and selection, this section focuses on how and when available information is consumed by the model. We distinguish two forms of data ingestion: training-time scheduling, which regulates the composition, weighting, and ordering of data during optimization, and runtime information use, which determines when and how external or self-generated information is incorporated during inference. Accordingly, data mixing, curriculum learning, and annealing are treated as training-time ingestion strategies, whereas retrieval, test-time search, decoding, and verification are treated as runtime information-use strategies. This distinction separates operations that modify data content or membership (Tier 2) from those that regulate its temporal or conditional use (Tier 3).

To illustrate how these ingestion strategies differ in practice, we compare the scheduling patterns reported for the LLaMA and DeepSeek lineages. The LLaMA lineage represents a highly structured, linear scheduling pipeline. It defaults to uniform dataset proportions during early optimization phases, but uniquely illustrates model capabilities by executing a massive, single-shot high-concentration data “annealing” phase during late-stage convergence. Conversely, the DeepSeek lineage adopts

Table 16 | Evidence-Informed Decision Matrix for Data Creation and Selection.

Methodology Paradigm	Core Mechanisms	Conditions of Applicability	Primary Failure Modes	Evidence-Informed Guidance
Reverse-KLD Distillation	White-box param matching; mode-seeking distribution optimization.	Restricted to open-source teacher-student pairs with explicit parameter visibility.	Vulnerable to vocabulary-level mode under-estimation; restricts creative diversity.	Consider for targeted distillation when teacher distributions are accessible; compare it with forward or mixed KLD objectives, and do not assume that it eliminates structural hallucinations.
Low-Direct-Annotation Generation	Prompt-free pre-query templates; self-play agent exploration.	Often applicable to vertical domains facing a strict human-annotated seed ceiling.	Accelerates semantic drift and cascading errors via ungrounded self-amplification.	Pair with external rule engines or expert-model discriminators (*Prometheus*) to break the hallucination loop.
Symbolic Logic Gen.	Code compilation; external solvers; prototype predicate trees.	Often useful for deterministic tasks with reliable executable or symbolic verifiers.	Rigid template constraints limit stylistic linguistic variety and natural dialogue tone.	Utilize a hybrid framework: synthesize reasoning traces via symbolic verifiers, then paraphrase via aligned text LLMs.
Gradient-Based Selection	Low-rank influence function; model-induced entropy filtering.	Tailored for high-efficiency, multi-task instruction fine-tuning runs.	Prohibitive computational overhead when scaling to trillion-token web-dumps.	Consider as a two-stage filter: prune web-dumps via cheap heuristics first, then apply gradient selection (*LESS*) on the coreset.

a highly non-linear, adaptive strategy. It implements multi-stage curriculum learning and model-aware dynamic mixture weights during baseline training, while aggressively scaling data trajectory scheduling into the post-training regime via online reinforcement learning and test-time search computational flywheels.

Figure 7 summarizes this distinction between training-time and runtime ingestion. During training, data organization, sequencing, and human-derived supervision regulate how learning signals are presented to the model, affecting effectiveness and efficiency. At inference time, self-generated or externally retrieved evidence can be incorporated to improve response reliability and truthfulness, although their effectiveness depends on the quality of the underlying evidence and verification process.

4.1. Efficiency & Effectiveness

Efficiency and effectiveness strategies govern the spatial composition of training batches and the sequential pipeline scheduling of tokens across the core optimization timeline.

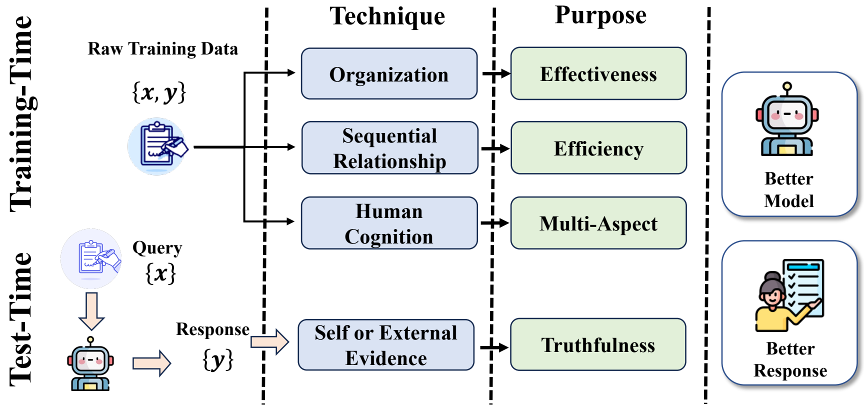

Figure 7 | Overview of data ingestion strategies across training and inference. Training-time strategies organize, sequence, and integrate learning signals to improve model effectiveness and efficiency, while runtime strategies incorporate self-generated or external evidence to improve response truthfulness.

4.1.1. Sample-Centric Strategies

This dimension focuses on how already available supervision is organized and presented during optimization. We distinguish between Training Sample Organization, which restructures or conditions existing inputs without necessarily creating persistent new supervision, and Data Mixture, which regulates the relative exposure of heterogeneous data sources during training. *1) Sample Construction* To optimize computational efficiency, Packing strategies [Dong et al., 2025b] concatenate multiple sequences into single samples, eliminating padding waste during training. Beyond formatting, construction methods actively steer cognitive behavior: Persona-Plug [Liu et al., 2025c] and advanced System Prompts [Lee et al., 2024b] embed explicit context embeddings to align generation with user intent, while CommonIT [Rao et al., 2024a] optimizes batch composition by clustering tasks with shared underlying logic. For robust preference learning, iterative methods like RFT [Dong et al., 2023, Zhu et al., 2024] and SeaPO [Rao et al., 2025a] construct training samples via rejection sampling or targeted error injection (negative contrast), creating a contrastive signal that sharpens the model's discrimination capabilities. RFT and SeaPO are cross-tier methods; here we discuss them specifically from the perspective of how dynamically constructed supervision is incorporated into optimization. *2) Sample Mixing* Determining the optimal ratio of heterogeneous data sources is critical to balancing breadth and depth. Moving beyond static heuristic ratios, dynamic approaches like DoReMi [Xie et al., 2023] and Predictive Data Selection [SHUM et al., 2025] utilize proxy models to calculate optimal mixture weights that minimize distribution shifts. Similarly, frameworks such as D4 [Tirumala et al., 2023], QuaDMix [Liu et al., 2025a], and SampleMix [Xi et al., 2025] explicitly optimize the trade-off between dataset diversity (deduplication) and quality (fidelity) to prevent redundancy. For expert models, mixture strategies focus on integrating new skills without catastrophic forgetting. Qwen2.5-Math [Yang et al., 2024a] employs a self-improving cycle of synthetic generation and structured filtering, while Baichuan-M2 [Team et al., 2025b] enforces rigid expert ratios (e.g., medical:general:math) to anchor domain expertise. Complementary methods like StatsMerging [Merugu et al., 2025] and Recycling the Web [Nguyen et al., 2025] leverage teacher distillation and discarded web corpora to maximize the information density of the training mix.

4.1.2. Training Pipeline Optimization

Training pipeline scheduling dictates the precise sequential order in which data paradigms are consumed across different model maturity phases.

1) Mid Training This phase acts as the critical cognitive bridge between large-scale pre-training and task-specific fine-tuning. "Annealing" strategies [Wang et al., 2025c, Wei et al., 2025, Zhang et al., 2025c] shift the data distribution from raw accumulation to structured reasoning by injecting instruction-following data early or utilizing high-quality synthetic mixtures. Capability Transition: Models like Phi [Abdin et al., 2024] and MiniCPM [Hu et al., 2024] demonstrate that mixing SFT data during this intermediate stage enables faster generalization. OLMo [OLMo et al., 2024] highlights the importance of curriculum annealing in learning rate schedules to stabilize this shift. RL Preparation: Specialized mid-training prepares models for complex reasoning. OctoThinker [Wang et al., 2025c] and RA3 [Zhang et al., 2025b] show that exposing models to long-chain reasoning or latent structure discovery during this phase is essential for unlocking the full potential of subsequent reinforcement learning (e.g., DeepSeek-R1 [Guo et al., 2025a]).

2) Multi-Stage Training In recent language model development, multi-stage training has evolved to emphasize high-quality data annealing through curriculum learning (CL) and replay strategies, enabling models to build capabilities incrementally while preserving critical knowledge. These approaches prioritize data quality gradients and strategic sample reuse to enhance learning efficiency and mitigate catastrophic forgetting.

Curriculum Learning Moving beyond random sampling, these strategies organize data by increasing difficulty, mirroring human pedagogy. Progressive Frameworks: Kimi K1.5 [Team et al., 2025a] employs a feedback-driven curriculum that evolves from basic pattern recognition to complex STEM reasoning, a strategy also adopted by Light-R1 [Wen et al., 2025] for SFT and DPO. Sorting Criteria: Complexity is quantified through diverse metrics: DELT [Dai et al., 2025] and DUMP [Wang et al., 2025d] sort samples by distribution-level learnability and loss gradients, VCRL [Jiang et al., 2025b] uses reward variance to calibrate RL difficulty, while other approaches leverage linguistic taxonomies[Ranaldi et al., 2024] or human educational levels [Lee et al., 2024a] to structure the learning path.

Replay Strategies To address the stability-plasticity dilemma (catastrophic forgetting), dynamic replay replaces simple shuffling. InsCL [Wang et al., 2024d] calculates Wasserstein distances to selectively replay tasks distinct from current inputs, ensuring broad coverage. Similarly, DMT [Dong et al., 2024] uses targeted replay to preserve specialized knowledge (e.g., coding) during general alignment, while SDFT [Yang et al., 2024c] and other memory-replay studies utilize self-distilled data to bridge distribution gaps and mitigate example-level forgetting. Liao et al. [2025] notes example-level forgetting in one-pass LLM pretraining. The lightweight memory replay eases forgetting, and training order is key.

4.2. Capability Alignment Strategies

Capability alignment strategies manage data scheduling during post-training fine-tuning and inference execution, transforming latent parametric knowledge into targeted, controllable behavioral outputs.

4.2.1. Constraint & Cognitive Alignment

Post-training alignment utilizes strict preference data scheduling to reshape the loss landscape. During Reinforcement Learning from Human/AI Feedback (RLHF/RLAIF) and DPO, data scheduling controls the contrastive preference margins. Advanced alignment schedules vary preference token distribution dynamically: presenting easy contrastive pairs early to establish stable style bounds, before shifting to hard adversarial contrastive pairs to reinforce safety and truthfulness metrics. Recent frameworks introduce self-aware dynamic preference sampling, where the model's real-time token entropy automatically dictates preference margin regularizers.

1) Constraint Acting as reliability gatekeepers, constraints prevent model drift and optimize the trade-off between plasticity and stability. Optimization Boundaries: To maintain coherence during adaptation, methods employ parameter-efficient tuning (LoRA [Hu et al., 2022]) and dynamic loss regularization [Rao et al., 2023a] via KL constraints [Rao et al., 2023b, Schulman et al., 2017], preventing the policy from collapsing into high-reward but low-fluency modes. Reward Shaping: Constraints are increasingly embedded directly into the objective function. SimPO [Meng et al., 2024] integrates length regularity to curb verbosity, KTO [Ethayarajh et al., 2024] leverages prospect theory to maximize utility without complex preference pairs, and DeepSeek-R1 [Guo et al., 2025a] utilizes verifiable correctness as a hard constraint to simplify reinforcement learning. Forgetting Mitigation: To constrain catastrophic forgetting, Function Vectors (FV) track Jiang et al. [2025a] and regularize the specific internal components responsible for task retention. However, the phenomenon of Spurious Forgetting[Zheng et al., 2025a], where “lost” capabilities spontaneously recover, suggests that training order and loss landscape dynamics impose implicit constraints that are still being characterized. *2) Cognitive Alignment* Moving beyond surface-level output matching, these strategies align training data with human cognitive architectures to foster robust reasoning. Theoretical Frameworks: CDT [Mo et al., 2025] integrates the Cattell-Horn-Carroll theory to map data onto a hierarchical framework of cognitive abilities, while CDS [Zhao et al., 2025b] utilizes cognitive diagnosis to detect specific knowledge deficits and generate targeted remedial data. Process-Oriented Learning: Alignment is shifting from outcome supervision to process supervision. Gandhi et al. [2025] demonstrate that the presence of specific reasoning behaviors (the “how”) is a stronger predictor of RL success than mere answer correctness (the “what”), suggesting that data strategies must explicitly cultivate these cognitive precursors.

4.2.2. Truthfulness & Consistency

The historical boundary between static training data and testing execution has collapsed into a continuous, active inference-time flywheel. When parametric weights are frozen, data strategy shifts to the runtime scheduling of test-time compute. DeepSeek-R1 demonstrates that reinforcement-learning-trained models can adaptively allocate more reasoning tokens to difficult problems at inference time. Simultaneously, inference-time data ingestion mechanisms like Contrastive Decoding (CAD) and dynamic Retrieval-Augmented Generation (RAG) adaptively alter token logit distributions at runtime and can mitigate factual errors by grounding generation in external evidence, although retrieval and evidence-use failures remain possible.

1) Consistency Ensuring factual reliability at test-time requires mitigating hallucinations arising from outdated training data or opaque generation logic. Strategies typically intervene at the consensus, retrieval, or token-decoding levels. Ensembling & Aggregation: Self-Consistency [Wang et al., 2023b] aggregates multiple reasoning paths to find a majority consensus, while Universal Self-Consistency (USC) [Chen et al., 2023c] extends this by employing the LLM itself as a semantic judge to select the most consistent output among candidates. Agentic & Verifiable Reasoning: Multi-Agent Debate [Du et al., 2024] improves factuality through iterative critique and consensus-building between agent instances. Moving toward intrinsic verification, Generative Verifiers [Zhang et al., 2025a] unify generation and verification objectives, while Self-Certainty [Kang et al., 2025] quantifies confidence directly from output statistics without external reward models. In multimodal contexts, MMBoundary [He et al., 2025c] calibrates reasoning-step confidence to reduce hallucinations in vision-language tasks.

2) RAG It mitigates hallucinations by anchoring generation in external evidence, though the integration itself requires optimization. Pipeline Optimization: HATHAT [Song et al., 2024] employs a hallucination-aware tuning pipeline to detect and rewrite errors, while SEER [Zhao et al., 2024c] optimizes evidence extraction using verifiable rewards for faithfulness and conciseness. Confidence &

Adaptation: TRAQ [Li et al., 2024d] integrates conformal prediction to provide statistical correctness guarantees for retrieved answers. Rowen [Ding et al., 2025] introduces adaptive retrieval, triggering external lookups only when the model's internal uncertainty exceeds a threshold, and Ayala and Bechard [2024] demonstrate that robust retrieval significantly reduces out-of-domain hallucinations.

3) Decoding This approach intervenes at the logit level, amplifying factual signals by contrasting distributions. Layer & Context Contrast: DOLA [Chuang et al., 2024] enhances factuality by contrasting logits between mature (higher) and premature (lower) transformer layers. CAD (Contrastive Decoding) [Shi et al., 2024b] adjusts probabilities by contrasting outputs generated with and without context, effectively isolating and amplifying context-aware tokens. Extensions: ROSE [Zhong et al., 2024] applies reverse prompt contrastive decoding to further suppress ungrounded content, while Waldendorf et al. [2024] extend this logic to multilingual translation by contrasting expert and amateur model distributions.

4.3. Summary: Temporal and Scheduling Methodologies

4.3.1. *The Plasticity-Stability Dilemma in Dynamic Mixing*

Data strategy at training-time is fundamentally a cybernetic regularization problem over the optimization timeline, rather than a static recipe formulation. A critical synthesis of empirical scaling trajectories shows that foundation models routinely suffer from the **plasticity-stability dilemma**: maximizing the ingestion speed of new domain knowledge routinely degrades generalized linguistic reasoning capabilities.

- **The Inefficiency of Static Mixing:** Fixed, heuristic dataset proportions may become suboptimal as model capabilities change and can contribute to forgetting in some settings, although they do not necessarily cause modal collapse. Because the target model's internal gradient landscape shifts dynamically as its capacity matures, static data mixtures introduce conflicting optimization vectors during late-stage convergence.
- **Dynamic Coordinate Optimization:** Dynamic paradigms attempt to address this by introducing proxy-model scaling or online gradient matching (e.g., *DoReMi, Predictive Selection*). Furthermore, recent 2025/2026 scheduling frameworks introduce **self-aware dynamic sampling** (e.g., utilizing the model's intrinsic confidence entropy to gauge instance complexity) and **strategic error amplification** to dynamically skew preference distribution as the model adapts, helping data mixtures remain better aligned to the model's immediate learnability frontier.

4.3.2. *Curriculum Progression and the Mechanics of Late-Stage Annealing*

Shifting from spatial arrangement to temporal sequencing, the ordering of data consumption plays a critical role in smooth optimization. The literature exhibits a paradigm shift from rigid simple-to-complex task ordering toward **multiphase capability transition via data annealing**:

- **Learnability Curriculums (Early-to-Mid Phase):** Structuring data ingestion based on distribution-level loss variance or gradient-norm difficulty metrics (e.g., *DEITA*, *DUMP*) smooths the initial non-convex loss landscape. By presenting regularized, low-entropy structures before injecting noisy, tail-end domain tasks, curriculum scheduling may improve early-stage convergence when the difficulty signal is well calibrated.
- **Late-Stage Annealing as a Cognitive Catalyst:** During the tail-end of pretraining or mid-adaptation phases, implementing a high-quality data "annealing" phase, characterized by a rapid

Table 17 | Evidence-Informed Decision Matrix for Temporal Data Strategies and Ingestion Scheduling.

Scheduling Paradigm	Core Mechanisms	Conditions of Applicability	Primary Failure Modes	Evidence-Informed Guidance
Dynamic Mixture Weighting	Online gradient tracking; proxy-model loss minimization (*DoReMi*).	Potentially useful for large-scale multi-domain pretraining and continuous pretraining.	High compute overhead due to simultaneous proxy training; sensitive to proxy architecture mismatch.	Consider dynamic weighting for large-scale multi-domain training when reliable proxy signals are available; the useful schedule is model- and domain-dependent.
Learnability Curriculum	Sample sorting by loss gradient; difficulty-aware dynamic windowing.	Tailored for domain-specific SFT cold-starts and high-efficiency mathematical alignment.	Risks catastrophic forgetting of early-stage general concepts if the difficulty threshold shifts too abruptly.	Consider retaining a replay component when forgetting is observed; the appropriate proportion depends on the distribution shift and training objective.
Late-Stage Data Annealing	Learning rate decay; exponential injection of high-fidelity CoT mixtures.	Commonly used in some model families for transforming a base pretraining model into a strong chat/reasoning candidate.	Over-annealing permanently erodes the model's raw plastic capacity, restricting future domain adaptation.	Use a limited late-stage phase selected through validation. The optimal token allocation and mixture composition are model-dependent.
Test-Time Compute Ingestion	Reflection-aware RL; dynamic MCTS search; inference verification loops.	Often useful for difficult or verifiable reasoning tasks.	Severe inference latency inflation; computationally prohibitive for real-time interactive chat applications.	Deploy a two-tier scheduling system: route routine queries to frozen parameters, and activate more expensive retrieval, search, or verification for queries with high estimated difficulty or uncertainty.

decay of the learning rate coupled with an exponential injection of high-density SFT, long-context text, and logical CoT sequences, has emerged as a common engineering practice in several model families (e.g., in state-of-the-art architectures like *OLMo*, *Llama-3*, and *Kimi K1.5*). This late-stage emphasis on higher-quality data may improve target capabilities and reduce the amount of subsequent alignment data required, although the mechanism and optimal allocation vary across model families.

4.3.3. The Post-Training Shift: From Offline Recipes to Online Inference Strategy

A growing research direction extends data-centric analysis beyond offline data preparation, which overlooks the temporal extension of data dynamics into the inference (*test-time*). The boundary between training-time data use and inference-time information acquisition has become increasingly interconnected:

- **Test-Time Compute Allocation:** As models freeze their parametric weights, data strategy shifts to the dynamic regulation of out-of-distribution environments. Through test-time verification, multi-turn Monte Carlo Tree Search (MCTS), and **reflection-aware online reinforcement learning** (e.g., employing lightweight companion reflection architectures), the lifecycle transitions from offline sample curation to the real-time runtime generation of search trajectories.
- **Dynamic Epistemic Ingestion:** Active inference mechanisms like Contrastive Decoding (*CAD*) and dynamic Retrieval-Augmented Generation (*RAG*) adaptively alter logit distribution at runtime. This runtime information acquisition can supplement parametric memory, but it does not eliminate its limitations, positioning inference-time verification not as an afterthought, but as a critical, final scheduling phase of the unified data lifecycle.

4.3.4. Evidence-Informed Decision Matrix

To translate these temporal insights into normative engineering protocols under varying compute budgets and optimization targets, Table 17 formalizes the applicability constraints, operational risks, and optimal execution paths for core scheduling paradigms. The table reports design options rather than universal recipes. The reported benefits depend on model scale, task distribution, proxy quality, and compute constraints.

5. Cross-Tier Synthesis: What the Lifecycle View Reveals

The value of the three-tier taxonomy lies not only in locating individual methods, but in exposing relationships that become less visible when datasets, synthesis, selection, training schedules, and agent systems are reviewed independently. Four recurring transitions emerge from the literature reviewed above.

First, data value is becoming model-conditional. Early curation pipelines largely optimize static properties of a corpus, including cleanliness, diversity, and heuristic quality. More recent selection, curriculum, and mixing methods increasingly condition data use on loss, gradients, confidence, errors, or the model's current capability state. From a lifecycle perspective, this connects Tier-2 selection with Tier-3 scheduling: the question changes from "Is this sample good?" to "Is this sample useful for this model at this point in training?"

Second, increasing synthesis autonomy relocates rather than eliminates human knowledge. As creation pipelines evolve from manual annotation to teacher generation, self-play, and autonomous loops, human input shifts from individual examples toward task definitions, verification criteria, environment design, and governance. The corresponding bottleneck therefore moves from annotation throughput toward the reliability of specifications and evaluators.

Third, agentic systems expand the definition of a data substrate. In conventional LLM training, data is predominantly a stored artifact. In emerging agentic pipelines, executable environments can themselves generate tasks, observations, failures, recovery paths, and reward signals. The data lifecycle consequently includes not only the curation of records but also the construction and validation of systems that produce records.

Fourth, data optimization increasingly continues after parameter training ends. Curriculum and mixture design regulate information ingestion while parameters are updated; retrieval, search, verification, and adaptive test-time computation regulate information ingestion after parameters are frozen. These mechanisms differ in optimization target, but they share the lifecycle-level problem of deciding what information should be consumed, when, and at what cost.

Together, these observations motivate a shift from treating "good data" as a fixed corpus toward treating data engineering as the joint design of artifacts, transformations, and adaptive information-use policies. This interpretation is the main analytical consequence of the proposed taxonomy.

6. Future Trends

The following directions are derived from recurrent gaps identified across the three lifecycle tiers rather than from an assumption that any single technique will universally improve LLM performance. We therefore distinguish trends already supported across multiple settings, such as increasing model awareness in data selection and greater use of executable verification, from more speculative directions such as fully autonomous data flywheels. Their effectiveness remains contingent on model scale, task distribution, verifier reliability, data provenance, and compute constraints.

6.1. Data Resources and Governance

6.1.1. Domain-Specific Data Scarcity

Sections 2.2 and 3.1.2 have highlighted that reasoning, safety and tool-use domains rely on manually crafted or model-augmented corpora whose absolute size is still orders of magnitude smaller than web text. Address scarcity in niche domains (e.g., low-resource languages, specialized science) by developing synthetic data strategies that encode domain-specific cognitive patterns. For example, use knowledge graphs [Li et al., 2025c] to generate synthetic data for rare medical conditions or leverage cross-lingual symbolic generation to build reasoning datasets for low-resource languages. Another promising direction is the construction of multimodal reasoning data for complex domain-specific scenarios [Zeng et al., 2025b], although its quality and faithfulness remain difficult to verify.

6.1.2. Cross-Domain Data Reutilization

Table 3-10 show that CoT rationales, symbolic expressions and tool-trajectories appear across maths, coding and agent benchmarks. Rather than re-annotating each field, we advocate lightweight transforms that repurpose these cognitive primitives for new domains, an approach that §3.1.2's structural-reuse paradigm has already hinted at. Identify domain-agnostic cognitive primitives (e.g., chain-of-thought rationales, symbolic expressions, tool-use trajectories) that carry general reasoning logic (step-by-step deduction, logical inference) across fields. Reusable structural patterns, such as symbolic derivations and tool-use trajectories, may support cross-domain transfer. However, transformed examples must be revalidated against the semantics, constraints, and evaluation criteria of the target domain. Surface-level substitution alone is unlikely to preserve validity.

6.1.3. Data Sensitivity and Privacy

The safety datasets in §2.2.2 and the privacy-oriented syntheses of §3.1.2 reveal an open tension: ethical reasoning requires realistic, high-stakes examples, yet real user records often cannot be directly released, centralized, or reused across organizations. Next-generation benchmarks should combine differential-privacy auditing, scenario-gated release and desensitisation that preserves logical structure to unlock sensitive domains without violating compliance. A central challenge is to preserve task-relevant reasoning signals while reducing privacy and compliance risks in high-stakes domains such as healthcare and finance. Develop fine-grained, data-scenario-risk evaluation frameworks that: (1) enforce interpretable outputs to verify ethical cognition; (2) conduct differential-privacy audits on residual training data; (3) implement dynamic scenario-gating to reduce misuse risks. Use desensitization techniques that preserve reasoning logic (e.g., generating synthetic but logically consistent medical data) rather than masking sensitive attributes, ensuring models learn domain-specific reasoning, not memorize answers.

6.2. Data Creation and Selection

6.2.1. Agentic Data Pipelines and Closed-Loop Flywheels

In taxonomy (§3), annotation, synthesis, and selection remain largely disjoint processes. To address this disconnect, future systems should embed these steps within adaptive agent loops: the model detects its own failures (as outlined in §3.1.2 on error-guided synthesis), triggers targeted data collection, and continuously updates filtering rules, ultimately moving toward the self-sustaining flywheel sketched below. Specifically, future work should develop adaptive agent systems that balance synthetic data generation (e.g., leveraging foundation models for domain-specific synthesis) with automated filtering (e.g., agent-driven bias detection), enabling efficient scaling of high-quality datasets. Some research has already begun exploring agents in complex automated scenarios, such as Deep Research [Li et al., 2025b]. Building on such advances, mechanisms should be implemented where LLMs identify their own erroneous outputs (e.g., through self-evaluation or validation against ground truth) and use these error samples to refine both data generation and filtering.

The iterative generate–filter–reuse pattern we identify in §4 can be closed into an autonomous data flywheel: initial seed data → train a small model → deploy to collect user feedback → generate harder synthetic cases → retrain. Such loops, already piloted in §3.1.2 seed-based generation, promise to reduce dependence on human annotation while keeping the distribution aligned with real-world demand. Such closed loops may also amplify model-generated errors, sampling bias, and reward misspecification, making external verification and human oversight necessary.

6.2.2. Multi-source Synthesis

§3.1.2 demonstrates that single-source expansion (e.g., Self-Instruct) risks domain drift. Intermediate and recursive fusion of heterogeneous corpora, web pages, textbooks, QA pairs, via knowledge-graph mediation (wikipedia [Kim et al., 2023] or tree structure [Maosongcao et al., 2025]) will be essential to maintain semantic coherence when scaling to new modalities or low-resource languages. Future work should investigate provenance-aware integration of heterogeneous sources, including entity alignment, conflict detection, source-reliability estimation, and cross-source verification. These mechanisms are needed to preserve semantic consistency when constructing data from web pages, textbooks, knowledge graphs, and multimodal resources.

6.2.3. User Collaboration

Current corpora are built either by experts or by fully automated scripts. §3.1.1's prompt-engineering workflows show that semi-expert users can validate or refine labels at scale. Future curation platforms should integrate federated learning and differential-privacy safeguards so that end-users become active contributors rather than passive data sources. Human collaboration, empowered by explicit user consent, will redefine data curation paradigms. Users will actively contribute to data refinement via interactive feedback loops, such as annotating ambiguous cases [Xu et al., 2025c] or validating model outputs [Kim et al., 2024]. Federated learning and differential privacy can reduce privacy risks under specified threat models, but neither guarantees the absence of leakage. User consent, access control, deletion mechanisms, and transparent governance therefore remain necessary. This transforms passive data subjects into active contributors, creating a dynamic ecosystem where user intent directly shapes model improvement.

6.2.4. Cross-modal Knowledge Utilization

§2.2.1 mentions visually grounded maths and chart QA, yet the bulk of discussed methods remain text-only. Unified text–vision embeddings and modal-agnostic symbolic representations are needed to extend the data-centric view to invoices, scanned notes, and slide decks, expanding the "reasoning primitives" concept beyond pure text. Data-centric LLMs will increasingly leverage cross-modal knowledge from structured forms, scanned documents, and image-text hybrids. Future work should focus on constructing, aligning, filtering, deduplicating, and verifying multimodal training data. Important challenges include preserving cross-modal correspondence, detecting contradictory evidence, controlling modality imbalance, and evaluating whether multimodal supervision improves transferable reasoning rather than only benchmark-specific perception.

6.2.5. Data Efficiency

Evidence reviewed in §4.1.2 suggests that curriculum design, mid-training, and replay can improve token efficiency under some settings. Information-density metrics, predictive-data selection (cf. §3.2.2) and small-model pilot experiments should be evaluated before large-scale training runs. Because conclusions from small models may not always transfer to larger models, pilot experiments should be combined with scaling-aware validation. Generating large-scale synthetic data (e.g., via LLMs) often introduces noise or bias, while rigorous selection to ensure quality can limit dataset size, creating a tension that hinders both generalization and task performance. Current approaches require massive data volumes across all stages [Bai et al., 2023], leading to high computational costs and environmental impact, with little emphasis on data efficiency. Data experiments on LLMs can be costly and have a low tolerance for errors [Lambert et al., 2024]. As a result, it is common practice to conduct initial experiments on smaller models [Xie et al., 2023]. It is important to design reasonable and feasible data experiments that evaluate data quality and information density, study data proportioning, and determine whether to introduce different data sets at different stages.

6.2.6. Length-Aware Data Synthesis and Selection

The survey repeatedly notes that over-long CoT or tool-trajectory sequences hurt readability and inference cost. Intent-length joint modelling, where the desired output span is predicted jointly with the target content, should be integrated into synthesis pipelines to curb verbosity at generation time rather than by post-hoc truncation. The core goal of Length Control is to constrain model outputs to task-appropriate lengths, eliminating overthinking-induced redundancy [Sui et al., 2025] (e.g., repetitive reasoning, irrelevant expansions) or under-expression.

6.2.7. Model-Aware Filtering

§3.2 shows filters that optimize universal quality scores can still select samples the current model already masters. Dynamic diagnostics, online error trackers, confidence surfaces, or lightweight probes, should gate the selection process, ensuring that each training batch targets the model's live weakness distribution instead of a static "ideal" set. Unlike current data selection (which primarily optimizes for universal metrics like quality or diversity), Model-Aware Filter centers on adapting data selection to a model's real-time capabilities–tailoring inputs to what the model currently needs, not just what is universally "good". Its core logic lies in dynamic alignment: for early-stage models (e.g., pre-trained but unfinetuned), it prioritizes foundational, low-complexity data to fill basic knowledge gaps; for mature models (e.g., domain-finetuned but error-prone in edge cases), it shifts to rare, high-complexity samples that target the model's specific weaknesses. It leverages lightweight "model capability diagnostics" [Rao et al., 2025a] (e.g., real-time error analysis [Cheng et al., 2024b], confidence scoring) to update selection rules iteratively. Model-aware filtering must also preserve exploration and coverage, because repeatedly selecting only high-loss or low-confidence samples may amplify noise, narrow the training distribution, or overfit transient model weaknesses.

6.3. Data Ingestion Strategies

6.3.1. Unified Post-Training

Figure 7 and §4.1.2 illustrate that transitions between separately optimized SFT and RL stages can introduce objective mismatch, policy drift, or repeated use of similar supervision. Single-stage objectives that fuse supervised signals with preference optimization, e.g., entropy-weighted reward or off-policy importance sampling, represent a promising direction to shorten the pipeline while preserving alignment. The traditional multi-stage iterative paradigm, limited by policy drift and efficiency loss, is gradually evolving toward single-stage unified approaches. Recent work explores more unified post-training objectives that combine supervised and preference-based signals. For example, SRFT [Fu et al., 2025] uses entropy-aware weighting to coordinate knowledge injection and policy refinement, while LUFFY [Yan et al., 2025] reuses SFT data as off-policy experience during RL. An important open question is how to balance these signals without introducing objective interference or training instability.

6.3.2. Cross-Stage Data Reutilization

We emphasise in §4 that the same corpus often undergoes reformatting per stage. Strategically reuse data across stages to strengthen cognitive consistency. For example, incorporate high-quality SFT data (well-annotated instructions) into pretrain [Allen-Zhu and Li, 2024] to seed task-specific knowledge early, or repurpose RL feedback data (human-preferred responses) as SFT examples to reinforce alignment [Bai et al., 2022]. This reduces redundant data collection and amplifies the impact of high-fidelity data on cognitive growth.

6.3.3. Data Unlearning

Finally, the privacy discussion in §4.1.2 is only half an answer if models cannot forget after revocation. Causal unlearning that surgically removes parameter contributions of specific sensitive examples, coupled with distribution-shift monitors, will be necessary to keep data-centric LLMs compliant without full retraining. Machine unlearning aims to reduce the influence of specified training examples or data categories without retraining [Shumailov et al., 2024], a model from scratch. However, reliably localizing and removing their effects remains an open problem [Yao et al., 2024] because information is distributed across parameters and may interact with related knowledge. Future evaluation should jointly measure forgetting efficacy, retained utility, privacy leakage, and robustness against relearning or extraction attacks.

6.3.4. Test-Time Strategy

§4.2.2 reviews consistency checks, RAG and contrastive decoding as post-hoc truthfulness layers. Calibrated confidence estimates may support adaptive routing of queries to different inference budgets, ranging from greedy decoding to retrieval and multi-sample verification. Future test-time strategies are converging toward confidence-guided and calibration-based adaptive inference. Recent advances such as CarBoN [Tang et al., 2025] refine Best-of-N sampling through calibrated logits and temperature tuning to improve efficiency, while Confidence-Informed Self-Consistency [Taubenfeld et al., 2025] prioritizes high-confidence reasoning paths, lowering the average sampling cost while maintaining accuracy. Remaining challenges include confidence miscalibration under distribution shift, unreliable uncertainty estimates, and the additional cost introduced by retrieval and verification.

7. Conclusion

This survey organizes the LLM data lifecycle according to three operational questions: what information-bearing artifacts are available, how those artifacts are created or selected, and when and how their information is consumed. The resulting Substrate–Creation/Selection–Ingestion decomposition is intended to complement, rather than replace, existing stage-based and technique-specific surveys. Applying this lens across pretraining, post-training, reasoning, safety, and agentic systems exposes several recurring transitions. Useful data is increasingly defined relative to the capability state of the target model rather than by static quality alone; increasing synthesis autonomy shifts human involvement toward specification and verification; executable environments extend the notion of data from stored artifacts to data-producing systems; and retrieval, search, and verification extend data optimization into inference-time information acquisition. These trends are not universal laws. Their benefits depend on model scale, task distribution, verification quality, and available compute, but together they indicate that modern data engineering increasingly concerns the coordination of artifacts, transformations, and adaptive information-use strategies. By making these distinctions explicit and linking them to setting-dependent evidence and failure modes, we hope the framework provides a useful basis for comparing data-centric methods and for identifying where future empirical evidence is most needed.

Funding Declaration

This work was supported in part by the Guangdong S&T Program (Grant No. 2024B0101050003), Guangdong Basic and Applied Basic Research Foundation (Grant No. 2024A1515011491), and Shenzhen Science and Technology Program (Grant Nos. ZDSYS20230626091203008,KJZD20231023094700001, KQTD2024072910215406). Thanks to the insights of ACL 25 tutorial on Synthetic Data in the Era of LLMs.

References

Marah Abdin, Jyoti Aneja, Harkirat Behl, Sébastien Bubeck, Ronen Eldan, Suriya Gunasekar, Michael Harrison, Russell J Hewett, Mojan Javaheripi, Piero Kauffmann, et al. Phi-4 technical report. *arXiv preprint arXiv:2412.08905*, 2024.

Rishabh Agarwal, Nino Vieillard, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. Gkd: Generalized knowledge distillation for auto-regressive sequence models. *ICLR*, 2024.

Aider AI. Aider: Ai-powered pair programming in your terminal. https://github.com/Aider-AI/aider, 2023. URL https://github.com/Aider-AI/aider.

Essential AI. Essential-web v1.0: 24t tokens of organized web data, 2025.

AI-MO. Aimo validation amc dataset on hugging face., 2023. URL https://huggingface.co/datasets/AI-MO/aimo-validationamc.

AI-MO. Aimo validation aime dataset on hugging face., 2024. URL https://huggingface.co/datasets/AI-MO/aimo-validationaime.

Alon Albalak, Yanai Elazar, Sang Michael Xie, Shayne Longpre, Nathan Lambert, Xinyi Wang, Niklas Muennighoff, Bairu Hou, Liangming Pan, Haewon Jeong, et al. A survey on data selection for language models. *Transactions on Machine Learning Research*, 2024.

Loubna Ben Allal, Raymond Li, Denis Kocetkov, Chenghao Mou, Christopher Akiki, Carlos Munoz Ferrandis, Niklas Muennighoff, Mayank Mishra, Alex Gu, Manan Dey, et al. Santacoder: don't reach for the stars!, 2023.

Bradley P. Allen, Prateek Chhikara, Thomas Macaulay Ferguson, Filip Ilievski, and Paul Groth. Sound and complete neurosymbolic reasoning with llm-grounded interpretations, 2025.

Zeyuan Allen-Zhu and Yuanzhi Li. Physics of language models: Part 3.1, knowledge storage and extraction, 2024.

Jacob Austin, Augustus Odena, Maxwell I. Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, and Charles Sutton. Program synthesis with large language models. *CoRR*, abs/2108.07732, 2021. URL https://arxiv.org/abs/2108.07732.

Orlando Ayala and Patrice Bechard. Reducing hallucination in structured outputs via retrieval-augmented generation. In *NAACL*, pages 228–238, June 2024.

Zhangir Azerbayev, Hailey Schoelkopf, Keiran Paster, Marco Dos Santos, Stephen Marcus McAleer, Albert Q. Jiang, Jia Deng, Stella Biderman, and Sean Welleck. Llemma: An open language model for mathematics. In *ICLR*, 2024.

Fei Bai, Huatong Song, Shuang Sun, Daixuan Cheng, Yike Yang, Chuan Hao, Renyuan Li, Feng Chang, Yuan Wei, Ran Tao, et al. Clawgym: A scalable framework for building effective claw agents. *arXiv preprint arXiv:2604.26904*, 2026.

Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional ai: Harmlessness from ai feedback, 2022.

Joachim Baumann, Vishakh Padmakumar, Xiang Li, John Yang, Diyi Yang, and Sanmi Koyejo. SWE-chat: Coding agent interactions from real users in the wild. *arXiv preprint arXiv:2604.20779*, 2026.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020.

Alexander Bukharin, Shiyang Li, Zhengyang Wang, Jingfeng Yang, Bing Yin, Xian Li, Chao Zhang, Tuo Zhao, and Haoming Jiang. Data diversity matters for robust instruction tuning. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *EMNLP-Findings*, pages 3411–3425, November 2024.

Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, Arjun Guha, Michael Greenberg, and Abhinav Jangda. Multipl-e: A scalable and extensible approach to benchmarking neural code generation, 2022.

Linzheng Chai, Shukai Liu, Jian Yang, Yuwei Yin, JinKe, Jiaheng Liu, Tao Sun, Ge Zhang, Changyu Ren, Hongcheng Guo, Noah Wang, Boyang Wang, Xianjie Wu, Bing Wang, Tongliang Li, Liqun Yang, Sufeng Duan, Zhaoxiang Zhang, and Zhoujun Li. Mceval: Massively multilingual code evaluation. In *ICLR*, 2025.

Amartya Chakraborty, Paresh Dashore, Nadia Bathaee, Anmol Jain, Anirban Das, Shi-Xiong Zhang, Sambit Sahu, Milind Naphade, and Genta Indra Winata. T1: A tool-oriented conversational dataset for multi-turn agentic planning, 2025.

Sahil Chaudhary. Code alpaca: An instruction-following llama model for code generation. https://github.com/sahil280114/codealpaca, 2023.

Hao Chen, Yiming Zhang, Qi Zhang, Hantao Yang, Xiaomeng Hu, Xuetao Ma, Yifan Yanggong, and Junbo Zhao. Maybe only 0.5 data is needed: A preliminary exploration of low training data instruction tuning, 2023a.

Jiaqi Chen, Tong Li, Jinghui Qin, Pan Lu, Liang Lin, Chongyu Chen, and Xiaodan Liang. UniGeo: Unifying geometry logical reasoning via reformulating mathematical expression. In *EMNLP*, pages 3313–3323, December 2022.

Lichang Chen, Shiyang Li, and Jun Yan. Alpagasus: Training a better alpaca model with fewer data. In *ICLR*, 2024a.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harrison Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Mingyang Chen, Haoze Sun, Tianpeng Li, Fan Yang, Hao Liang, Keer Lu, Bin Cui, Wentao Zhang, Zenan Zhou, and Weipeng Chen. Facilitating multi-turn function calling for LLMs via compositional instruction tuning. In *ICLR*, 2025a.

Wenhu Chen, Ming Yin, Max Ku, Pan Lu, Yixin Wan, Xueguang Ma, Jianyu Xu, Xinyi Wang, and Tony Xia. TheoremQA: A theorem-driven question answering dataset. In *EMNLP*, pages 7889–7901, December 2023b.

Xinyun Chen, Renat Aksitov, Uri Alon, Jie Ren, Kefan Xiao, Pengcheng Yin, Sushant Prakash, Charles Sutton, Xuezhi Wang, and Denny Zhou. Universal self-consistency for large language model generation, 2023c.

Xuanzhong Chen, Zile Qiao, Guoxin Chen, Liangcai Su, Zhen Zhang, Xinyu Wang, Pengjun Xie, Fei Huang, Jingren Zhou, and Yong Jiang. Agentfrontier: Expanding the capability frontier of LLM agents with ZPD-guided data synthesis. *arXiv preprint arXiv:2510.24695*, 2025b.

Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. Self-play fine-tuning converts weak language models to strong language models. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net, 2024b.

Daixuan Cheng, Yuxian Gu, Shaohan Huang, Junyu Bi, Minlie Huang, and Furu Wei. Instruction pre-training: Language models are supervised multitask learners. In *EMNLP*, pages 2529–2550, 2024a.

Jiale Cheng, Yida Lu, Xiaotao Gu, Pei Ke, Xiao Liu, Yuxiao Dong, Hongning Wang, Jie Tang, and Minlie Huang. AutoDetect: Towards a unified framework for automated weakness detection in large language models. In *EMNLP-Findings*, pages 6786–6803, 2024b. doi: 10.18653/v1/2024.findings-emnlp.397.

Joshua Chiu, Partha Protim Paul, and Zahin Wahab. Enhancing leakage attacks on searchable symmetric encryption using llm-based synthetic data generation. *arXiv preprint arXiv:2504.20414*, 2025.

Yung-Sung Chuang, Yujia Xie, Hongyin Luo, Yoon Kim, James R. Glass, and Pengcheng He. Dola: Decoding by contrasting layers improves factuality in large language models. In *ICLR*, 2024.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Mike Conover, Matt Hayes, Ankit Mathur, Jianwei Xie, Jun Wan, Sam Shah, Ali Ghodsi, Patrick Wendell, Matei Zaharia, and Reynold Xin. Free dolly: Introducing the world's first truly open instruction-tuned llm, 2023.

Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Wei Zhu, Yuan Ni, Guotong Xie, Zhiyuan Liu, and Maosong Sun. UltraFeedback: Boosting language models with high-quality feedback. In *ICML*, 2024.

Yalun Dai, Yangyu Huang, Xin Zhang, Wenshan Wu, Chong Li, Wenhui Lu, Shijie Cao, Li Dong, and Scarlett Li. Data efficacy for language model training, 2025.

Hanxing Ding, Liang Pang, Zihao Wei, Huawei Shen, and Xueqi Cheng. Rowen: Adaptive retrieval-augmented generation for hallucination mitigation in llms, 2025.

Yangruibo Ding, Zijian Wang, Wasi Uddin Ahmad, Hantian Ding, Ming Tan, Nihal Jain, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, and Bing Xiang. Crosscodeeval: A diverse and multilingual benchmark for cross-file code completion. In *NeurIPS*, 2023.

Guanting Dong, Hongyi Yuan, Keming Lu, Chengpeng Li, Mingfeng Xue, Dayiheng Liu, Wei Wang, Zheng Yuan, Chang Zhou, and Jingren Zhou. How abilities in large language models are affected by supervised fine-tuning data composition. In *ACL*, pages 177–198, 2024.

Guanting Dong, Keming Lu, Chengpeng Li, Tingyu Xia, Bowen Yu, Chang Zhou, and Jingren Zhou. Self-play with execution feedback: Improving instruction-following capabilities of large language models. In *The Thirteenth International Conference on Learning Representations*, 2025a.

Guanting Dong, Junting Lu, Junjie Huang, Wanjun Zhong, Longxiang Liu, Shijue Huang, Zhenyu Li, Yang Zhao, Xiaoshuai Song, Xiaoxi Li, et al. Agent-world: Scaling real-world environment synthesis for evolving general agent intelligence. *arXiv preprint arXiv:2604.18292*, 2026.

Hanze Dong, Wei Xiong, Deepanshu Goyal, Yihan Zhang, Winnie Chow, Rui Pan, Shizhe Diao, Jipeng Zhang, KaShun SHUM, and Tong Zhang. RAFT: Reward ranked finetuning for generative foundation model alignment. *Transactions on Machine Learning Research*, 2023. ISSN 2835-8856.

Jiancheng Dong, Lei Jiang, Wei Jin, and Lu Cheng. Threshold filtering packing for supervised fine-tuning: Training related samples within packs. In *NAACL*, pages 4422–4435, April 2025b. ISBN 979-8-89176-189-6.

Qianlong Du, Chengqing Zong, and Jiajun Zhang. Mods: Model-oriented data selection for instruction tuning, 2023.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *Forty-first International Conference on Machine Learning*, 2024.

Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Yann Dubois, Chen Xuechen Li, Rohan Taori, Tianyi Zhang, Ishaan Gulrajani, Jimmy Ba, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Alpacafarm: A simulation framework for methods that learn from human feedback. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine, editors, *NeurIPS*, 2024.

Alex Egg, Martin Iglesias Goyanes, Friso Kingma, Andreu Mora, Leandro von Werra, and Thomas Wolf. Dabstep: Data agent benchmark for multi-step reasoning, 2025.

Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, and Douwe Kiela. Model alignment as prospect theoretic optimization. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

Ivan Evtimov, Arman Zharmagambetov, Aaron Grattafiori, Chuan Guo, and Kamalika Chaudhuri. WASP: Benchmarking web agent security against prompt injection attacks. In *NeurIPS*, 2025.

Run-Ze Fan, Xuefeng Li, Haoyang Zou, Junlong Li, Shwai He, Ethan Chern, Jiewen Hu, and Pengfei Liu. Reformatted alignment. In *EMNLP-Findings*, pages 574–597, November 2024.

Meng Fang, Shilong Deng, Yudi Zhang, Zijing Shi, Ling Chen, Mykola Pechenizkiy, and Jun Wang. Large language models are neurosymbolic reasoners. In *Proceedings of the AAAI conference on artificial intelligence*, pages 17985–17993, 2024.

James D. Finch and Jinho D. Choi. Diverse and effective synthetic data generation for adaptable zero-shot dialogue state tracking. In *EMNLP-Findings*, pages 12527–12544, November 2024.

Yuqian Fu, Tinghong Chen, Jiajun Chai, Xihuai Wang, Songjun Tu, Guojun Yin, Wei Lin, Qichao Zhang, Yuanheng Zhu, and Dongbin Zhao. Srft: A single-stage method with supervised and reinforcement fine-tuning for reasoning, 2025.

Kanishk Gandhi, Ayush K Chakravarthy, Anikait Singh, Nathan Lile, and Noah Goodman. Cognitive behaviors that enable self-improving reasoners, or, four habits of highly effective STars. In *Second Conference on Language Modeling*, 2025.

Saumya Gandhi, Ritu Gala, Vijay Viswanathan, Tongshuang Wu, and Graham Neubig. Better synthetic data by retrieving and transforming existing datasets. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *ACL-Findings*, pages 6453–6466, August 2024.

Tao Ge, Xin Chan, Xiaoyang Wang, Dian Yu, Haitao Mi, and Dong Yu. Scaling synthetic data creation with 1,000,000,000 personas, 2025.

Hajar Emami Gohari, Swanand Ravindra Kadhe, Syed Yousaf Shah, Constantin Adam, Abdulhamid Adebayo, Praneet Adusumilli, Farhan Ahmed, Nathalie Baracaldo Angel, Santosh Subhashrao Borse, Yuan-Chi Chang, et al. Gneissweb: Preparing high quality data for llms at scale, 2025.

Arya Grayeli, Atharva Sehgal, Omar Costilla-Reyes, Miles Cranmer, and Swarat Chaudhuri. Symbolic regression with a learned concept library. In *NeurIPS*, volume 37, 2024.

Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida I. Wang. Cruxeval: A benchmark for code reasoning, understanding and execution. In *ICML*, 2024a.

Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. Minillm: Knowledge distillation of large language models. In *ICLR*, 2024b.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025a.

Yiduo Guo, Zhen Guo, Chuanwei Huang, Zi-Ang Wang, Zekai Zhang, Haofei Yu, Huishuai Zhang, and Yikang Shen. Synthetic data rl: Task definition is all you need, 2025b.

Seungju Han, Kavel Rao, Allyson Ettinger, Liwei Jiang, Bill Yuchen Lin, Nathan Lambert, Yejin Choi, and Nouha Dziri. Wildguard: Open one-stop moderation tools for safety risks, jailbreaks, and refusals of llms. In *NeurIPS*, 2025.

Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. OlympiadBench: A challenging benchmark for promoting AGI with olympiad-level bilingual multimodal scientific problems. In *ACL*, pages 3828–3850, August 2024.

Feng He, Zijun Chen, Xinnian Liang, Tingting Ma, Yunqi Qiu, Shuangzhi Wu, and Junchi Yan. Protoreasoning: Prototypes as the foundation for generalizable reasoning in llms, 2025a.

Yanheng He, Jiahe Jin, and Pengfei Liu. Efficient agent training for computer use. *arXiv preprint arXiv:2505.13909*, 2025b.

Zhitao He, Sandeep Polisetty, Zhiyuan Fan, Yuchen Huang, Shujin Wu, and Yi R. Fung. Mmboundary: Advancing mllm knowledge boundary awareness through reasoning step confidence calibration, 2025c.

Zhiwei He, Tian Liang, Jiahao Xu, Qiuzhi Liu, Xingyu Chen, Yue Wang, Linfeng Song, Dian Yu, Zhenwen Liang, Wenxuan Wang, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. Deepmath-103k: A large-scale, challenging, decontaminated, and verifiable mathematical dataset for advancing reasoning, 2025d.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS Datasets and Benchmarks*, 2021.

Namgyu Ho, Laura Schmid, and Se-Young Yun. Large language models are reasoning teachers. In *ACL*, pages 14852–14882, July 2023.

Or Honovich, Thomas Scialom, Omer Levy, and Timo Schick. Unnatural instructions: Tuning language models with (almost) no human labor. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14409–14428, Toronto, Canada, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.806.

Cheng-Yu Hsieh, Chun-Liang Li, Chih-kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alex Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Findings of the Association for Computational Linguistics: ACL 2023*, pages 8003–8017, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-acl.507.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *ICLR*, 2022.

Jinchao Hu, Meizhi Zhong, Kehai Chen, Xuefeng Bai, and Min Zhang. Agentic tool use in large language models. *arXiv preprint arXiv:2604.00835*, 2026.

Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Weilin Zhao, Xinrong Zhang, Zheng Leng Thai, Kaihuo Zhang, Chongyi Wang, Yuan Yao, Chenyang Zhao, Jie Zhou, Jie Cai, Zhongwu Zhai, Ning Ding, Chao Jia, Guoyang Zeng, Dahai Li, Zhiyuan Liu, and Maosong Sun. Minicpm: Unveiling the potential of small language models with scalable training strategies, 2024.

Yue Huang, Jiawen Shi, Yuan Li, Chenrui Fan, Siyuan Wu, Qihui Zhang, Yixin Liu, Pan Zhou, Yao Wan, Neil Zhenqiang Gong, et al. Metatool benchmark for large language models: Deciding whether to use tools and which to use. In *ICLR*, 2024a.

Yue Huang, Lichao Sun, Haoran Wang, Siyuan Wu, Qihui Zhang, Yuan Li, Chujie Gao, Yixin Huang, Wenhan Lyu, Yixuan Zhang, et al. TrustLLM: Trustworthiness in large language models. In *ICML*, volume 235, pages 20166–20270, 2024b. URL https://proceedings.mlr.press/v235/huang24x.html.

Hamish Ivison, Yizhong Wang, Jiacheng Liu, Zeqiu Wu, Valentina Pyatkin, Nathan Lambert, Noah A Smith, Yejin Choi, and Hannaneh Hajishirzi. Unpacking DPO and PPO: Disentangling best practices for learning from preference feedback. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *The Thirteenth International Conference on Learning Representations*, 2025.

Gangwei Jiang, Caigao JIANG, Zhaoyi Li, Siqiao Xue, JUN ZHOU, Linqi Song, Defu Lian, and Ying Wei. Unlocking the power of function vectors for characterizing and mitigating catastrophic forgetting in continual instruction tuning. In *The Thirteenth International Conference on Learning Representations*, 2025a.

Guochao Jiang, Wenfeng Feng, Guofeng Quan, Chuzhan Hao, Yuewei Zhang, Guohua Liu, and Hao Wang. Vcrl: Variance-based curriculum reinforcement learning for large language models, 2025b.

Liwei Jiang, Kavel Rao, Seungju Han, Allyson Ettinger, Faeze Brahman, Sachin Kumar, Niloofar Mireshghallah, Ximing Lu, Maarten Sap, Yejin Choi, and Nouha Dziri. Wildteaming at scale: From in-the-wild jailbreaks to (adversarially) safer language models. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

Ashvini Jindal. Arithmo-mistral-7b: Mathematical reasoning model. Hugging Face, 2023.

Jaehun Jung, Peter West, Liwei Jiang, Faeze Brahman, Ximing Lu, Jillian Fisher, Taylor Sorensen, and Yejin Choi. Impossible distillation for paraphrasing and summarization: How to make high-quality lemonade out of small, low-quality model. In *NAACL*, pages 4439–4454, June 2024.

Jaehun Jung, Seungju Han, Ximing Lu, Skyler Hallinan, David Acuna, Shrimai Prabhumoye, Mostafa Patwary, Mohammad Shoeybi, Bryan Catanzaro, and Yejin Choi. Prismatic synthesis: Gradient-based data diversification boosts generalization in llm reasoning, 2025.

Zhewei Kang, Xuandong Zhao, and Dawn Song. Scalable best-of-n selection for large language models via self-certainty. In *NeurIPS*, 2025.

Xiaopeng Ke, Hexuan Deng, Xuebo Liu, Jun Rao, Zhenxi Song, Jun Yu, and Min Zhang. Aquilt: Weaving logic and self-inspection into low-cost, high-relevance data synthesis for specialist llms. In *EMNLP*, 2025.

Muhammad Khalifa, Lajanugen Logeswaran, Moontae Lee, Honglak Lee, and Lu Wang. Exploring demonstration ensembling for in-context learning, 2023.

Hyunwoo Kim, Jack Hessel, Liwei Jiang, Peter West, Ximing Lu, Youngjae Yu, Pei Zhou, Ronan Bras, Malihe Alikhani, Gunhee Kim, Maarten Sap, and Yejin Choi. SODA: Million-scale dialogue distillation with social commonsense contextualization. In *EMNLP*, pages 12930–12949, December 2023.

Seungone Kim, Juyoung Suk, Shayne Longpre, Bill Yuchen Lin, Jamin Shin, Sean Welleck, Graham Neubig, Moontae Lee, Kyungjae Lee, and Minjoon Seo. Prometheus 2: An open source language model specialized in evaluating other language models. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *EMNLP*, pages 4334–4353, Miami, Florida, USA, 2024. doi: 10.18653/v1/2024.emnlp-main.248.

Yoon Kim and Alexander M. Rush. Sequence-level knowledge distillation. In Jian Su, Kevin Duh, and Xavier Carreras, editors, *EMNLP*, pages 1317–1327, Austin, Texas, November 2016. Association for Computational Linguistics. doi: 10.18653/v1/D16-1139.

Abdullatif Köksal, Timo Schick, Anna Korhonen, and Hinrich Schuetze. LongForm: Effective instruction tuning with reverse instructions. In *EMNLP-Findings*, pages 7056–7078, November 2024.

Rik Koncel-Kedziorski, Subhro Roy, Aida Amini, Nate Kushman, and Hannaneh Hajishirzi. MAWPS: A math word problem repository. In *NAACL*, pages 1152–1157, June 2016.

Alexey Kurakin, Natalia Ponomareva, Umar Syed, Liam MacDermed, and Andreas Terzis. Harnessing large-language models to generate private synthetic text. *arXiv preprint arXiv:2306.01684*, 2023.

Abdullatif Köksal, Marion Thaler, Ayyoob Imani, Ahmet Üstün, Anna Korhonen, and Hinrich Schütze. Muri: High-quality instruction tuning datasets for low-resource languages via reverse instructions, 2024.

Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, et al. Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.

Bruce W Lee, Hyunsoo Cho, and Kang Min Yoo. Instruction tuning with human curriculum. In *NAACL-Findings*, pages 1281–1309, June 2024a.

Seongyun Lee, Sue Hyun Park, Seungone Kim, and Minjoon Seo. Aligning to thousands of preferences via system message generalization. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024b.

Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In *EMNLP*, pages 3045–3059, November 2021.

Chengpeng Li, Zheng Yuan, Hongyi Yuan, Guanting Dong, Keming Lu, Jiancan Wu, Chuanqi Tan, Xiang Wang, and Chang Zhou. MuggleMath: Assessing the impact of query and response augmentation on math reasoning. In *ACL*, pages 10230–10258, August 2024a.

Haoran Li, Qingxiu Dong, Zhengyang Tang, Chaojun Wang, Xingxing Zhang, Haoyang Huang, Shaohan Huang, Xiaolong Huang, Zeqiang Huang, Dongdong Zhang, Yuxian Gu, Xin Cheng, Xun Wang, Si-Qing Chen, Li Dong, Wei Lu, Zhifang Sui, Benyou Wang, Wai Lam, and Furu Wei. Synthetic data (almost) from scratch: Generalized instruction tuning for language models, 2024b.

Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Yitzhak Gadre, Hritik Bansal, Etash Guha, Sedrick Scott Keh, Kushal Arora, et al. Datacomp-lm: In search of the next generation of training sets for language models, 2025a.

Jiachun Li, Zhuoran Jin, Tianyi Men, Yupu Hao, Kejian Zhu, Lingshuai Wang, Dongqi Huang, Longxiang Wang, Shengjia Hua, Lu Wang, et al. Agentic environment engineering for large language models: A survey of environment modeling, synthesis, evaluation, and application. *arXiv preprint arXiv:2606.12191*, 2026a.

Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C.C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. In *NeurIPS*, 2023a.

Kuan Li, Zhongwang Zhang, Huifeng Yin, Rui Ye, Yida Zhao, Liwen Zhang, Litu Ou, Dingchu Zhang, Xixi Wu, Jialong Wu, Xinyu Wang, Zile Qiao, Zhen Zhang, Yong Jiang, Pengjun Xie, Fei Huang, and Jingren Zhou. Websailor-v2: Bridging the chasm to proprietary agents via synthetic data and scalable reinforcement learning, 2025b.

Kuan Li, Zhongwang Zhang, Huifeng Yin, Liwen Zhang, Litu Ou, Jialong Wu, Wenbiao Yin, Baixuan Li, Zhengwei Tao, Xinyu Wang, Weizhou Shen, Junkai Zhang, Dingchu Zhang, Xixi Wu, Yong Jiang, Ming Yan, Pengjun Xie, Fei Huang, and Jingren Zhou. Websailor: Navigating super-human reasoning for web agent, 2025c.

Lan Li, Liri Fang, Bertram Ludäscher, and Vetle I. Torvik. Autodcworkflow: Llm-based data cleaning workflow auto-generation and benchmark. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025d.

Ming Li, Yong Zhang, Zhitao Li, Jiuhai Chen, Lichang Chen, Ning Cheng, Jianzong Wang, Tianyi Zhou, and Jing Xiao. From quantity to quality: Boosting LLM performance with self-guided data selection for instruction tuning. In *NAACL*, pages 7602–7635, June 2024c.

Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-bank: A comprehensive benchmark for tool-augmented LLMs. In *EMNLP*, pages 3102–3116, December 2023b.

Shuo Li, Sangdon Park, Insup Lee, and Osbert Bastani. TRAQ: Trustworthy retrieval augmented question answering via conformal prediction. In *NAACL*, pages 3799–3821, June 2024d.

Tianle Li, Wei-Lin Chiang, Evan Frick, Lisa Dunlap, Banghua Zhu, Joseph E. Gonzalez, and Ion Stoica. From live data to high-quality benchmarks: The Arena-Hard pipeline, 2024e.

Xian Li, Ping Yu, Chunting Zhou, Timo Schick, Omer Levy, Luke Zettlemoyer, Jason E Weston, and Mike Lewis. Self-alignment with instruction backtranslation. In *The Twelfth International Conference on Learning Representations*, 2024f.

Xiaoyuan Li, Wenjie Wang, Moxin Li, Junrong Guo, Yang Zhang, and Fuli Feng. Evaluating mathematical reasoning of large language models: A focus on error identification and correction. In *ACL-Findings*, pages 11316–11360, August 2024g.

Xirui Li, Ming Li, Ion Stoica, Cho-Jui Hsieh, and Tianyi Zhou. Clawenvkit: Automatic environment generation for claw-like agents. *arXiv preprint arXiv:2604.18543*, 2026b.

Xuechen Li, Tianyi Zhang, Yann Dubois, Rohan Taori, Ishaan Gulrajani, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Alpacaeval: An automatic evaluator of instruction-following models. https://github.com/tatsu-lab/alpaca_eval, 2023c.

Wing Lian, Bleys Goodson, Eugene Pentland, Austin Cook, Chanvichet Vong, and "Teknium". Openorca: An open dataset of gpt augmented flan reasoning traces. https://https://huggingface.co/Open-Orca/OpenOrca, 2023.

Xiao Liang, Zhong-Zhi Li, Yeyun Gong, Yang Wang, Hengyuan Zhang, Yelong Shen, Ying Nian Wu, and Weizhu Chen. Sws: Self-aware weakness-driven problem synthesis in reinforcement learning for llm reasoning, 2025.

Yijuan Liang, Xinghao Chen, Yifan Ge, Ziyi Wu, Hao Wu, Changyu Zeng, Wei Xing, and Xiaoyu Shen. Unitoolcall: Unifying tool-use representation, data, and evaluation for LLM agents. *arXiv preprint arXiv:2604.11557*, 2026.

Chonghua Liao, Ruobing Xie, Xingwu Sun, Haowen Sun, and Zhanhui Kang. Exploring forgetting in large language model pre-training. In *ACL*, pages 2112–2127, July 2025. ISBN 979-8-89176-251-0.

Minpeng Liao, Chengxi Li, Wei Luo, Wu Jing, and Kai Fan. MARIO: MAth reasoning with code interpreter output - a reproducible pipeline. In *ACL-Findings*, pages 905–924, August 2024.

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *The Twelfth International Conference on Learning Representations*, 2024.

Stephanie Lin, Jacob Hilton, and Owain Evans. TruthfulQA: Measuring how models mimic human falsehoods. In *ACL*, pages 3214–3252, 2022.

Wang Ling, Dani Yogatama, Chris Dyer, and Phil Blunsom. Program induction by rationale generation: Learning to solve and explain algebraic word problems. In *ACL*, pages 158–167, July 2017.

Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024a.

Fengze Liu, Weidong Zhou, Binbin Liu, Zhimiao Yu, Yifan Zhang, Haobin Lin, Yifeng Yu, Bingni Zhang, Xiaohuan Zhou, Taifeng Wang, and Yong Cao. Quadmix: Quality-diversity balanced data selection for efficient llm pretraining, 2025a.

Haoxiong Liu, Yifan Zhang, Yifan Luo, and Andrew Chi-Chih Yao. Augmenting math word problems via iterative question composing. In *AAAI*, 2025b.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. In *NeurIPS*, 2023.

Jiongnan Liu, Yutao Zhu, Shuting Wang, Xiaochi Wei, Erxue Min, Yu Lu, Shuaiqiang Wang, Dawei Yin, and Zhicheng Dou. LLMs + persona-plug = personalized LLMs. In *ACL*, pages 9373–9385, July 2025c. ISBN 979-8-89176-251-0.

Liangxin Liu, Xuebo Liu, Derek F. Wong, Dongfang Li, Ziyi Wang, Baotian Hu, and Min Zhang. SelectIT: Selective instruction tuning for LLMs via uncertainty-aware self-reflection. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024b.

Ruibo Liu, Jerry Wei, Fangyu Liu, Chenglei Si, Yanzhe Zhang, Jinmeng Rao, Steven Zheng, Daiyi Peng, Diyi Yang, Denny Zhou, and Andrew M. Dai. Best practices and lessons learned on synthetic data for language models. In *Conference on Language Modeling*, 2024c.

Shukai Liu, Linzheng Chai, Jian Yang, Jiajun Shi, He Zhu, Liran Wang, Ke Jin, Wei Zhang, Hualei Zhu, Shuyue Guo, Tao Sun, Jiaheng Liu, Yunlong Duan, Yu Hao, Liqun Yang, Guanglin Niu, Ge Zhang, and Zhoujun Li. Mdeval: Massively multilingual code debugging, 2025d.

Wei Liu, Weihao Zeng, Keqing He, Yong Jiang, and Junxian He. What makes good data for alignment? a comprehensive study of automatic data selection in instruction tuning. In *The Twelfth International Conference on Learning Representations*, 2024d.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. In *The Twelfth International Conference on Learning Representations*, 2024e.

Zuxin Liu, Thai Quoc Hoang, Jianguo Zhang, Ming Zhu, Tian Lan, Shirley Kokane, Juntao Tan, Weiran Yao, Zhiwei Liu, Yihao Feng, Rithesh R N, Liangwei Yang, Silvio Savarese, Juan Carlos Niebles, Huan Wang, Shelby Heinecke, and Caiming Xiong. APIGen: Automated PIpeline for generating verifiable and diverse function-calling datasets. In *NeurIPS*, 2024f.

Lin Long, Rui Wang, Ruixuan Xiao, Junbo Zhao, Xiao Ding, Gang Chen, and Haobo Wang. On LLMs-driven synthetic data generation, curation, and evaluation: A survey. *arXiv preprint arXiv:2406.15126*, 2024.

Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Haoping Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. In *NAACL-Findings*, pages 1160–1183, April 2025. ISBN 979-8-89176-195-7.

Keming Lu, Hongyi Yuan, Zheng Yuan, Runji Lin, Junyang Lin, Chuanqi Tan, Chang Zhou, and Jingren Zhou. #instag: Instruction tagging for analyzing supervised fine-tuning of large language models. In *The Twelfth International Conference on Learning Representations*, 2024a.

Pan Lu, Hritik Bansal, Tony Xia, Jiacheng Liu, Chunyuan Li, Hannaneh Hajishirzi, Hao Cheng, Kai-Wei Chang, Michel Galley, and Jianfeng Gao. Mathvista: Evaluating mathematical reasoning of foundation models in visual contexts. In *ICLR*, 2024b.

Yao Lu, Max Bartolo, Alastair Moore, Sebastian Riedel, and Pontus Stenetorp. Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity. In *ACL*, pages 8086–8098, May 2022.

Zimu Lu, Aojun Zhou, Houxing Ren, Ke Wang, Weikang Shi, Junting Pan, Mingjie Zhan, and Hongsheng Li. MathGenie: Generating synthetic data with question back-translation for enhancing mathematical reasoning of LLMs. In *ACL*, pages 2732–2747, August 2024c.

Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. Wizardcoder: Empowering code large language models with evol-instruct. In *ICLR*, 2024.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *NeurIPS*, volume 36, 2023.

Pratyush Maini, Skyler Seto, Richard Bai, David Grangier, Yizhe Zhang, and Navdeep Jaitly. Rephrasing the web: A recipe for compute and data-efficient language modeling. In *ACL*, pages 14044–14072, August 2024.

Chaitanya Manem, Pratik Prabhanjan Brahma, Prakamya Mishra, Zicheng Liu, and Emad Barsoum. Sand-math: Using llms to generate novel, difficult and useful mathematics questions and answers, 2025.

Maosongcao Maosongcao, Taolin Zhang, Mo Li, Chuyu Zhang, Yunxin Liu, Conghui He, Haodong Duan, Songyang Zhang, and Kai Chen. Condor: Enhance LLM alignment with knowledge-driven data synthesis and refinement. In *ACL*, pages 22392–22412, July 2025. ISBN 979-8-89176-251-0.

Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. ChartQA: A benchmark for question answering about charts with visual and logical reasoning. In *ACL-Findings*, pages 2263–2279, May 2022.

Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. Harmbench: a standardized evaluation framework for automated red teaming and robust refusal. In *ICML*, 2024.

Yu Meng, Mengzhou Xia, and Danqi Chen. SimPO: Simple preference optimization with a reference-free reward. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

Ranjith Merugu, Bryan Bo Cao, and Shubham Jain. Statsmerging: Statistics-guided model merging via task-specific teacher distillation, 2025.

Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. Gaia: a benchmark for general ai assistants. In *The Twelfth International Conference on Learning Representations*, 2024.

Swaroop Mishra, Matthew Finlayson, Pan Lu, Leonard Tang, Sean Welleck, Chitta Baral, Tanmay Rajpurohit, Oyvind Tafjord, Ashish Sabharwal, Peter Clark, and Ashwin Kalyan. LILA: A unified benchmark for mathematical reasoning. In *EMNLP*, pages 5807–5832, December 2022.

Arindam Mitra, Luciano Del Corro, Guoqing Zheng, Shweti Mahajan, Dany Rouhana, Andres Codas, Yadong Lu, Wei-ge Chen, Olga Vrousgos, Corby Rosset, et al. Agentinstruct: Toward generative teaching with agentic flows. *arXiv preprint arXiv:2407.03502*, 2024.

Haosi Mo, Xinyu Ma, Xuebo Liu, Derek F. Wong, Yu Li, Jie Liu, and Min Zhang. Cdt: A comprehensive capability framework for large language models across cognition, domain, and task. In *EMNLP-Findings*, 2025.

Niklas Muennighoff, Thomas Wang, Lintang Sutawika, Adam Roberts, Stella Biderman, Teven Le Scao, M Saiful Bari, Sheng Shen, Zheng Xin Yong, Hailey Schoelkopf, et al. Crosslingual generalization through multitask finetuning. In *ACL*, pages 15991–16111, 2023.

Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling, 2025.

Supriya Nagesh, Justin Y Chen, Nina Mishra, and Tal Wagner. Private text generation by seeding large language model prompts. *arXiv preprint arXiv:2502.13193*, 2025.

Md Mahadi Hasan Nahid and Sadid Bin Hasan. Safesynthdp: Leveraging large language models for privacy-preserving synthetic data generation using differential privacy. *arXiv preprint arXiv:2412.20641*, 2024.

Nihal Nayak, Yiyang Nan, Avi Trost, and Stephen Bach. Learning to generate instruction tuning datasets for zero-shot task adaptation. In *ACL-Findings*, pages 12585–12611, August 2024.

Joseph P Near, David Darais, Naomi Lefkovitz, Gary S Howarth, et al. *Guidelines for evaluating differential privacy guarantees*. US Department of Commerce, National Institute of Standards and Technology, 2025.

Nex-AGI Team. Nex-n1: Agentic models trained via a unified ecosystem for large-scale environment construction. *arXiv preprint arXiv:2512.04987*, 2025.

Sina Bagheri Nezhad and Ameeta Agrawal. Enhancing large language models with neurosymbolic reasoning for multilingual tasks, 2025.

Thao Nguyen, Jeffrey Li, Sewoong Oh, Ludwig Schmidt, Jason E Weston, Luke Zettlemoyer, and Xian Li. Better alignment with instruction back-and-forth translation. In *EMNLP-Findings*, pages 13289–13308, November 2024.

Thao Nguyen, Yang Li, Olga Golovneva, Luke Zettlemoyer, Sewoong Oh, Ludwig Schmidt, and Xian Li. Recycling the web: A method to enhance pre-training data quality and quantity for language models. In *Second Conference on Language Modeling*, 2025.

Songjie Niu, Kaisen Yang, Rui Zhao, Yichao Liu, Zonglin Li, Hongning Wang, and Wenguang Chen. Tree-KG: An expandable knowledge graph construction framework for knowledge-intensive domains. In *ACL*, pages 18516–18529, July 2025. doi: 10.18653/v1/2025.acl-long.907.

Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, et al. 2 olmo 2 furious. *arXiv preprint arXiv:2501.00656*, 2024.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *NeurIPS*, volume 35, pages 27730–27744, 2022.

Liangming Pan, Alon Albalak, Xinyi Wang, and William Wang. Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning. In *EMNLP-Findings*, pages 3806–3824, December 2023.

Zhuoshi Pan, Yu Li, Honglin Lin, Qizhi Pei, Zinan Tang, Wei Wu, Chenlin Ming, H. Vicky Zhao, Conghui He, and Lijun Wu. LEMMA: Learning from errors for MatheMatical advancement in LLMs. In *ACL-Findings*, pages 11615–11639, July 2025. ISBN 979-8-89176-256-5.

Jinlong Pang, Na Di, Zhaowei Zhu, Jiaheng Wei, Hao Cheng, Chen Qian, and Yang Liu. Token cleaning: Fine-grained data selection for LLM supervised fine-tuning. In *Forty-second International Conference on Machine Learning*, 2025.

Ajay Patel, Colin Raffel, and Chris Callison-Burch. DataDreamer: A tool for synthetic data generation and reproducible LLM workflows. In *ACL*, pages 3781–3799, August 2024.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis. In *Advances in Neural Information Processing Systems*, 2024.

Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *ICML*, 2025.

Baolin Peng, Chunyuan Li, Pengcheng He, Michel Galley, and Jianfeng Gao. Instruction tuning with GPT-4. *arXiv preprint arXiv:2304.03277*, 2023.

Ru Peng, Kexin Yang, Yawen Zeng, Junyang Lin, Dayiheng Liu, and Junbo Zhao. Dataman: Data manager for pre-training large language models. In *The Thirteenth International Conference on Learning Representations*, 2025.

Haritz Puerto, Martin Tutek, Somak Aditya, Xiaodan Zhu, and Iryna Gurevych. Code prompting elicits conditional reasoning abilities in Text+Code LLMs. In *EMNLP*, pages 11234–11258, November 2024.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *The Twelfth International Conference on Learning Representations*, 2024.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine, editors, *NeurIPS*, 2023.

Leonardo Ranaldi and Andre Freitas. Self-refine instruction-tuning for aligning reasoning in language models. In *EMNLP*, pages 2325–2347, November 2024.

Leonardo Ranaldi, Giulia Pucci, and Andrè Freitas. Does the *Order* matter? Curriculum learning over languages. In *LREC-COLING*, pages 5212–5220, May 2024.

Jun Rao, Liang Ding, Shuhan Qi, Meng Fang, Yang Liu, Li Shen, and Dacheng Tao. Dynamic contrastive distillation for image-text retrieval. *IEEE Transactions on Multimedia*, pages 1–13, 2023a.

Jun Rao, Xv Meng, Liang Ding, Shuhan Qi, Xuebo Liu, Min Zhang, and Dacheng Tao. Parameter-efficient and student-friendly knowledge distillation. *IEEE Trans. Multim.*, pages 1–12, 2023b.

Jun Rao, Xuebo Liu, Lian Lian, Shengjun Cheng, Yunjie Liao, and Min Zhang. CommonIT: Commonality-aware instruction tuning for large language models via data partitions. In *EMNLP*, pages 10064–10083, 2024a.

Jun Rao, Xuebo Liu, Zepeng Lin, Liang Ding, Jing Li, Dacheng Tao, and Min Zhang. Exploring and enhancing the transfer of distribution in knowledge distillation for autoregressive language models, 2024b.

Jun Rao, Yunjie Liao, Xuebo Liu, Zepeng Lin, Lian Lian, Dong Jin, Shengjun Cheng, Jun Yu, and Min Zhang. Seapo: Strategic error amplification for robust preference optimization of large language models. In *EMNLP-Findings*, 2025a.

Jun Rao, Zepeng Lin, Xuebo Liu, Xiaopeng Ke, Lian Lian, Dong Jin, Shengjun Cheng, Jun Yu, and Min Zhang. APT: Improving specialist LLM performance with weakness case acquisition and iterative preference training. In *ACL-Findings*, pages 20958–20980, July 2025b. ISBN 979-8-89176-256-5.

Paul Röttger, Hannah Kirk, Bertie Vidgen, Giuseppe Attanasio, Federico Bianchi, and Dirk Hovy. XSTest: A test suite for identifying exaggerated safety behaviours in large language models. In *NAACL*, pages 5377–5400, June 2024.

William Saunders, Catherine Yeh, Jeff Wu, Steven Bills, Long Ouyang, Jonathan Ward, and Jan Leike. Self-critiquing models for assisting human evaluators. *arXiv preprint arXiv:2206.05802*, 2022.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *NeurIPS*, 2023.

Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, Aayush Gupta, HyoJung Han, Sevien Schulhoff, et al. The prompt report: A systematic survey of prompt engineering techniques, 2025.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Amrith Setlur, Saurabh Garg, Xinyang Geng, Naman Garg, Virginia Smith, and Aviral Kumar. RL on incorrect synthetic data scales the efficiency of LLM math reasoning by eight-fold. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

Xingyu Shen, Shengding Hu, Xinrong Zhang, Xu Han, Xiaojun Meng, Jiansheng Wei, Zhiyuan Liu, and Maosong Sun. AutoClean: LLMs can prepare their training corpus. In *NAACL*, pages 85–95, April 2025a. ISBN 979-8-89176-191-9.

Xinyue Shen, Zeyuan Chen, Michael Backes, Yun Shen, and Yang Zhang. "do anything now": Characterizing and evaluating in-the-wild jailbreak prompts on large language models. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, page 1671–1685, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400706363. doi: 10.1145/3658644.3670388.

Zhanming Shen, Hao Chen, Yulei Tang, Shaolin Zhu, Wentao Ye, Xiaomeng Hu, Haobo Wang, Gang Chen, and Junbo Zhao. Cycle-instruct: Fully seed-free instruction tuning via dual self-training and cycle consistency, 2025b.

Taiwei Shi, Kai Chen, and Jieyu Zhao. Safer-instruct: Aligning language models with automated preference data. In *NAACL*, pages 7636–7651, June 2024a.

Weijia Shi, Xiaochuang Han, Mike Lewis, Yulia Tsvetkov, Luke Zettlemoyer, and Wen-tau Yih. Trusting your evidence: Hallucinate less with context-aware decoding. In *NAACL*, pages 783–791, June 2024b.

KaShun SHUM, Yuzhen Huang, Hongjian Zou, dingqi, YiXuan Liao, Xiaoxin Chen, Qian Liu, and Junxian He. Predictive data selection: The data that predicts is the data that teaches. In *Forty-second International Conference on Machine Learning*, 2025.

Ilia Shumailov, Jamie Hayes, Eleni Triantafillou, Guillermo Ortiz-Jimenez, Nicolas Papernot, Matthew Jagielski, Itay Yona, Heidi Howard, and Eugene Bagdasaryan. Ununlearning: Unlearning is not sufficient for content regulation in advanced generative ai. *arXiv preprint arXiv:2407.00106*, 2024.

Damien Sileo. Scaling synthetic logical reasoning datasets with context-sensitive declarative grammars. In *EMNLP*, pages 5275–5283, November 2024.

Daria Soboleva, Faisal Al-Khateeb, Robert Myers, Jacob R Steeves, Joel Hestness, and Nolan Dey. SlimPajama: A 627B token cleaned and deduplicated version of RedPajama, 2023.

Luca Soldaini, Rodney Kinney, Akshita Bhagia, Dustin Schwenk, David Atkinson, Russell Authur, Ben Bogin, Khyathi Chandu, Jennifer Dumas, Yanai Elazar, et al. Dolma: An Open Corpus of Three Trillion Tokens for Language Model Pretraining Research. In *ACL*, August 2024.

Guijin Son, Seungyeop Yi, Minju Gwak, Hyunwoo Ko, Wongi Jang, and Youngjae Yu. Researchmath-14k: Scaling research-level mathematics via agents. *arXiv preprint arXiv:2605.28003*, 2026.

Juntong Song, Xingguang Wang, Juno Zhu, Yuanhao Wu, Xuxin Cheng, Randy Zhong, and Cheng Niu. RAG-HAT: A hallucination-aware tuning pipeline for LLM in retrieval-augmented generation. In *EMNLP*, pages 1548–1558, November 2024.

Yang Sui, Yu-Neng Chuang, Guanchu Wang, Jiamu Zhang, Tianyi Zhang, Jiayi Yuan, Hongyi Liu, Andrew Wen, Shaochen Zhong, Na Zou, Hanjie Chen, and Xia Hu. Stop overthinking: A survey on efficient reasoning for large language models. *TMLR*, 2025. ISSN 2835-8856.

Zhiqing Sun, Yikang Shen, Qinhong Zhou, Hongxin Zhang, Zhenfang Chen, David Daniel Cox, Yiming Yang, and Chuang Gan. Principle-driven self-alignment of language models from scratch with minimal human supervision. In *NeurIPS*, 2023.

Zhiqing Sun, Yikang Shen, Hongxin Zhang, Qinhong Zhou, Zhenfang Chen, David Daniel Cox, Yiming Yang, and Chuang Gan. SALMON: Self-alignment with instructable reward models. In *The Twelfth International Conference on Learning Representations*, 2024.

Qiaoyu Tang, Ziliang Deng, Hongyu Lin, Xianpei Han, Qiao Liang, Boxi Cao, and Le Sun. Toolalpaca: Generalized tool learning for language models with 3000 simulated cases. *arXiv preprint arXiv:2306.05301*, 2023.

Yung-Chen Tang, Pin-Yu Chen, and Andrea Cavallaro. Carbon: Calibrated best-of-n sampling improves test-time reasoning, 2025.

Zhengyang Tang, Xingxing Zhang, Benyou Wang, and Furu Wei. Mathscale: Scaling instruction tuning for mathematical reasoning. In *ICML*, pages 47885–47900, 2024.

Amir Taubenfeld, Tom Sheffer, Eran Ofek, Amir Feder, Ariel Goldstein, Zorik Gekhman, and Gal Yona. Confidence improves self-consistency in llms. In *ACL-Findings*, page 20090–20111, 2025.

Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, et al. Kimi k1. 5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*, 2025a.

M2 Team, Chengfeng Dou, Chong Liu, Fan Yang, Fei Li, Jiyuan Jia, Mingyang Chen, Qiang Ju, Shuai Wang, Shunya Dang, Tianpeng Li, Xiangrong Zeng, and et al. Yijie Zhou. Baichuan-m2: Scaling medical capability with large verifier system, 2025b.

Kushal Tirumala, Daniel Simig, Armen Aghajanyan, and Ari S. Morcos. D4: Improving LLM pretraining via document de-duplication and diversification. In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2023.

Shubham Toshniwal, Ivan Moshkov, Sean Narenthiran, Daria Gitman, Fei Jia, and Igor Gitman. Openmathinstruct-1: A 1.8 million math instruction tuning dataset. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017.

Jonas Waldendorf, Barry Haddow, and Alexandra Birch. Contrastive decoding reduces hallucinations in large multilingual machine translation models. In *EACL*, pages 2526–2539, March 2024.

Jiachen T. Wang, Tong Wu, Dawn Song, Prateek Mittal, and Ruoxi Jia. Greats: Online selection of high-quality data for llm training in every iteration. In *NeurIPS*, volume 37, pages 131197–131223, 2024a.

Ke Wang, Jiahui Zhu, Minjie Ren, Zeming Liu, Shiwei Li, Zongye Zhang, Chenkai Zhang, Xiaoyu Wu, Qiqi Zhan, Qingjie Liu, and Yunhong Wang. A survey on data synthesis and augmentation for large language models. *arXiv preprint arXiv:2410.12896*, 2024b.

Ruida Wang, Wangchunshu Zhou, and Mrinmaya Sachan. Let's synthesize step by step: Iterative dataset synthesis with large language models by extrapolating errors from small models. In *EMNLP-Findings*, pages 11817–11831, December 2023a.

Shuhe Wang, Guoyin Wang, Yizhong Wang, Jiwei Li, Eduard Hovy, and Chen Guo. Packing analysis: Packing is more appropriate for large models or datasets in supervised fine-tuning. In *ACL-Findings*, pages 4953–4967, July 2025a.

Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better llm agents. In *ICML*, 2024c.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models, 2023b.

Yifan Wang, Yafei Liu, Chufan Shi, Haoling Li, Chen Chen, Haonan Lu, and Yujiu Yang. InsCL: A data-efficient continual learning paradigm for fine-tuning large language models with instructions. In *NAACL*, pages 663–677, June 2024d.

Yizhong Wang, Swaroop Mishra, Pegah Alipoormolabashi, Yeganeh Kordi, Amirreza Mirzaei, Atharva Naik, Arjun Ashok, Arut Selvan Dhanasekaran, Anjana Arunkumar, David Stap, et al. Super-NaturalInstructions: Generalization via declarative instructions on 1600+ NLP tasks. In *EMNLP*, 2022.

Yizhong Wang, Hamish Ivison, Pradeep Dasigi, Jack Hessel, Tushar Khot, Khyathi Raghavi Chandu, David Wadden, Kelsey MacMillan, Noah A. Smith, Iz Beltagy, et al. How far can camels go? exploring the state of instruction tuning on open resources. In *NeurIPS*, 2023c.

Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. In *ACL*, pages 13484–13508, 2023d.

Yudong Wang, Zixuan Fu, Jie Cai, Peijun Tang, Hongya Lyu, Yewei Fang, Zhi Zheng, Jie Zhou, Guoyang Zeng, Chaojun Xiao, Xu Han, and Zhiyuan Liu. Ultra-fineweb: Efficient data filtering and verification for high-quality llm training data, 2025b.

Yuqing Wang and Yun Zhao. Metacognitive prompting improves understanding in large language models. In *NAACL*, pages 1914–1926, June 2024.

Yuxin Wang, Duanyu Feng, Yongfu Dai, Zhengyu Chen, Jimin Huang, Sophia Ananiadou, Qianqian Xie, and Hao Wang. Harmonic: Harnessing llms for tabular data synthesis and privacy protection. *Advances in Neural Information Processing Systems*, 37:100196–100212, 2024e.

Zengzhi Wang, Fan Zhou, Xuefeng Li, and Pengfei Liu. Octothinker: Mid-training incentivizes reinforcement learning scaling, 2025c.

Zhenting Wang, Guofeng Cui, Yu-Jhe Li, Kun Wan, and Wentian Zhao. Dump: Automated distribution-level curriculum learning for rl-based llm post-training, 2025d.

Zilong Wang, Hao Zhang, Chun-Liang Li, Julian Martin Eisenschlos, Vincent Perot, Zifeng Wang, Lesly Miculicich, Yasuhisa Fujii, Jingbo Shang, Chen-Yu Lee, and Tomas Pfister. Chain-of-table: Evolving tables in the reasoning chain for table understanding. In *ICLR*, 2024f.

Maurice Weber, Daniel Y. Fu, Quentin Anthony, Yonatan Oren, Shane Adams, Anton Alexandrov, Xiaozhong Lyu, Huu Nguyen, Xiaozhe Yao, Virginia Adams, Ben Athiwaratkun, Rahul Chalamala, Kezhen Chen, Max Ryabinin, Tri Dao, Percy Liang, Christopher Ré, Irina Rish, and Ce Zhang. Redpajama: an open dataset for training large language models. *NeurIPS Datasets and Benchmarks Track*, 2024.

Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le. Finetuned language models are zero-shot learners. In *ICLR*. OpenReview.net, 2022.

Tianwen Wei, Jian Luan, Wei Liu, Shuang Dong, and Bin Wang. Cmath: Can your language model pass chinese elementary school math test?, 2023a.

Tianxin Wei, Ting-Wei Li, Zhining Liu, Xuying Ning, Ze Yang, Jiaru Zou, Zhichen Zeng, Ruizhong Qiu, Xiao Lin, Dongqi Fu, et al. Agentic reasoning for large language models. *arXiv preprint arXiv:2601.12538*, 2026.

Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. Magicoder: Source code is all you need. In *ICML*, 2023b.

Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I. Wang. Swe-rl: Advancing llm reasoning via reinforcement learning on open software evolution, 2025.

Liang Wen, Yunke Cai, Fenrui Xiao, Xin He, Qi An, Zhenyu Duan, Yimin Du, Junchen Liu, Lifu Tang, Xiaowei Lv, Haosheng Zou, Yongchao Deng, Shousheng Jia, and Xiangzheng Zhang. Light-r1: Curriculum sft, dpo and rl for long cot from scratch and beyond, 2025.

Jules White, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C. Schmidt. A prompt pattern catalog to enhance prompt engineering with chatgpt, 2023.

Di Wu, Wasi Uddin Ahmad, Dejiao Zhang, Murali Krishna Ramanathan, and Xiaofei Ma. Repoformer: Selective retrieval for repository-level code completion. In *ICML*, 2024.

Shujin Wu, Cheng Qian, Yi R. Fung, Paul Pu Liang, and Heng Ji. Alice: Proactive learning with teacher's demonstrations for weak-to-strong generalization, 2025.

Xiangyu Xi, Deyang Kong, Jian Yang, Jiawei Yang, Zhengyu Chen, Wei Wang, Jingang Wang, Xunliang Cai, Shikun Zhang, and Wei Ye. Samplemix: A sample-wise pre-training data mixing strategey by coordinating data quality and diversity. *arXiv preprint arXiv:2503.01506*, 2025.

Mengzhou Xia, Sadhika Malladi, Suchin Gururangan, Sanjeev Arora, and Danqi Chen. Less: Selecting influential data for targeted instruction tuning. In *International Conference on Machine Learning*, pages 54104–54132. PMLR, 2024a.

Tingyu Xia, Bowen Yu, Kai Dang, An Yang, Yuan Wu, Yuan Tian, Yi Chang, and Junyang Lin. Rethinking data selection at scale: Random selection is almost all you need, 2024b.

Zikai Xiao, Jianhong Tu, Chuhang Zou, Yuxin Zuo, Zhi Li, Peng Wang, Bowen Yu, Fei Huang, Junyang Lin, and Zuozhu Liu. Webworld: A large-scale world model for web agent training. *arXiv preprint arXiv:2602.14721*, 2026.

Chulin Xie, Zinan Lin, Arturs Backurs, Sivakanth Gopi, Da Yu, Huseyin A Inan, Harsha Nori, Haotian Jiang, Huishuai Zhang, Yin Tat Lee, et al. Differentially private synthetic data via foundation model apis 2: Text. In *ICML*, 2024.

Sang Michael Xie, Hieu Pham, Xuanyi Dong, Nan Du, Hanxiao Liu, Yifeng Lu, Percy Liang, Quoc V Le, Tengyu Ma, and Adams Wei Yu. Doremi: Optimizing data mixtures speeds up language model pretraining. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.

Kai Xiong, Xiao Ding, Yixin Cao, Yuxiong Yan, Li Du, Yufei Zhang, Jinglong Gao, Jiaqian Liu, Bing Qin, and Ting Liu. Com^2 : A causal-guided benchmark for exploring complex commonsense reasoning in large language models. In *ACL*, pages 16119–16140, July 2025. ISBN 979-8-89176-251-0.

Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, Qingwei Lin, and Daxin Jiang. Wizardlm: Empowering large pre-trained language models to follow complex instructions. In *ICLR*, 2023a.

Canwen Xu, Daya Guo, Nan Duan, and Julian McAuley. Baize: An open-source chat model with parameter-efficient tuning on self-chat data. In *EMNLP*, pages 6268–6278, 2023b.

Frank F. Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Z. Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, et al. Theagentcompany: Benchmarking LLM agents on consequential real world tasks. In *NeurIPS*, 2025a.

Jundong Xu, Hao Fei, Liangming Pan, Qian Liu, Mong-Li Lee, and Wynne Hsu. Faithful logical reasoning via symbolic chain-of-thought. In *ACL*, pages 13326–13365, August 2024.

Jundong Xu, Hao Fei, Meng Luo, Qian Liu, Liangming Pan, William Yang Wang, Preslav Nakov, Mong-Li Lee, and Wynne Hsu. Aristotle: Mastering logical reasoning with a logic-complete decompose-search-resolve framework. In *ACL*, pages 3052–3075, July 2025b. ISBN 979-8-89176-251-0.

Kaishuai Xu, Tiezheng Yu, Wenjun Hou, Yi Cheng, Chak Tou Leong, Liangyou Li, Xin Jiang, Lifeng Shang, Qun Liu, and Wenjie Li. Subtle errors in reasoning: Preference learning via error-injected self-editing. In *ACL*, pages 31184–31203, July 2025c. ISBN 979-8-89176-251-0.

Yang Xu, Yongqiang Yao, Yufan Huang, Mengnan Qi, Maoquan Wang, Bin Gu, and Neel Sundaresan. Rethinking the instruction quality: Lift is what you need, 2023c.

Zengzhuang Xu, Bingguang Hao, Zechuan Wang, Yuntao Wen, Maolin Wang, Yang Liu, Long Chen, Yicheng Chen, Cunyin Peng, Chenyi Zhuang, Jinjie Gu, Leilei Gan, Xiangyu Zhao, and Shi Gu. Funreason-mt technical report: Overcoming the complexity barrier in real-world multi-turn function calling. *arXiv preprint arXiv:2510.24645*, 2025d.

Zhangchen Xu, Fengqing Jiang, Luyao Niu, Yuntian Deng, Radha Poovendran, Yejin Choi, and Bill Yuchen Lin. Magpie: Alignment data synthesis from scratch by prompting aligned llms with nothing. In *ICLR*, 2025e.

Zhangchen Xu, Adriana Meza Soria, Shawn Tan, Anurag Roy, Ashish Sunil Agrawal, Radha Poovendran, and Rameswar Panda. TOUCAN: Synthesizing 1.5m tool-agentic data from real-world MCP environments. *arXiv preprint arXiv:2510.01179*, 2025f.

Fuzhao Xue, Yao Fu, Wangchunshu Zhou, Zangwei Zheng, and Yang You. To repeat or not to repeat: Insights from scaling llm under token-crisis. In *NeurIPS*, volume 36, pages 59304–59322, 2023.

Jianhao Yan, Yafu Li, Zican Hu, Zhi Wang, Ganqu Cui, Xiaoye Qu, Yu Cheng, and Yue Zhang. Learning to reason under off-policy guidance, 2025.

An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, Keming Lu, Mingfeng Xue, Runji Lin, Tianyu Liu, Xingzhang Ren, and Zhenru Zhang. Qwen2.5-math technical report: Toward mathematical expert model via self-improvement, 2024a.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *The Twelfth International Conference on Learning Representations*, 2024b.

Kaiyu Yang, Aidan M Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. Leandojo: Theorem proving with retrieval-augmented language models. In *NeurIPS*, 2023.

Zhaorui Yang, Tianyu Pang, Haozhe Feng, Han Wang, Wei Chen, Minfeng Zhu, and Qian Liu. Self-distillation bridges distribution gap in language model fine-tuning. In *ACL*, pages 1028–1043, 2024c.

Zitong Yang, Neil Band, Shuangping Li, Emmanuel Candes, and Tatsunori Hashimoto. Synthetic continued pretraining. In *ICLR*, 2025.

Yuanshun Yao, Xiaojun Xu, and Yang Liu. Large language model unlearning. In *NeurIPS*, volume 37, pages 105425–105475, 2024.

Qinyuan Ye, Mohamed Ahmed, Reid Pryzant, and Fereshte Khani. Prompt engineering a prompt engineer. In *ACL-Findings*, pages 355–385, August 2024.

Yixin Ye, Zhen Huang, Yang Xiao, Ethan Chern, Shijie Xia, and Pengfei Liu. Limo: Less is more for reasoning. In *COLM*, 2025.

Fan Yin, Zifeng Wang, I-Hung Hsu, Jun Yan, Ke Jiang, Yanfei Chen, Jindong Gu, Long Le, Kai-Wei Chang, Chen-Yu Lee, Hamid Palangi, and Tomas Pfister. Magnet: Multi-turn tool-use data synthesis and distillation via graph translation. In *ACL*, pages 32600–32616, July 2025. ISBN 979-8-89176-251-0.

Erxin Yu, Jing Li, Ming Liao, Qi Zhu, Boyang Xue, Minghui Xu, Baojun Wang, Lanqing Hong, Fei Mi, and Lifeng Shang. Self-error-instruct: Generalizing from errors for LLMs mathematical reasoning. In *ACL*, pages 8504–8519, July 2025. ISBN 979-8-89176-251-0.

Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T. Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. Metamath: Bootstrap your own mathematical questions for large language models. In *ICLR*, 2023.

Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *EMNLP*, 2018.

Zhaojian Yu, Xin Zhang, Ning Shang, Yangyu Huang, Can Xu, Yishujie Zhao, Wenxiang Hu, and Qiufeng Yin. Wavecoder: Widespread and versatile enhancement for code large language models by instruction tuning. In *ACL*, pages 5140–5153, 2024a.

Zichun Yu, Spandan Das, and Chenyan Xiong. Mates: Model-aware data selection for efficient pretraining with data influence models. In *NeurIPS*, volume 37, pages 108735–108759, 2024b.

Xiang Yue, Xingwei Qu, Ge Zhang, Yao Fu, Wenhao Huang, Huan Sun, Yu Su, and Wenhu Chen. MAmmoTH: Building math generalist models through hybrid instruction tuning. In *The Twelfth International Conference on Learning Representations*, 2024a.

Xiang Yue, Tuney Zheng, Ge Zhang, and Wenhu Chen. Mammoth2: Scaling instructions from the web. In *NeurIPS*, 2024b.

Daoguang Zan, Ailun Yu, Wei Liu, Dong Chen, Bo Shen, Wei Li, Yafen Yao, Ying Lu, Qianxiang Wang, and Yongji Wang. Multi-swe-bench: A multilingual benchmark for issue resolving. In *NeurIPS*, 2025.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah Goodman. STar: Bootstrapping reasoning with reasoning. In *NeurIPS*, 2022.

Aohan Zeng, Mingdao Liu, Rui Lu, Bowen Wang, Xiao Liu, Yuxiao Dong, and Jie Tang. AgentTuning: Enabling generalized agent abilities for LLMs. In *ACL-Findings*, pages 3053–3077, August 2024.

Aohan Zeng, Xin Lv, Qinkai Zheng, Zhenyu Hou, Bin Chen, Chengxing Xie, Cunxiang Wang, Da Yin, Hao Zeng, Jiajie Zhang, et al. Glm-4.5: Agentic, reasoning, and coding (arc) foundation models. *arXiv preprint arXiv:2508.06471*, 2025a.

Shuang Zeng, Xinyuan Chang, Mengwei Xie, Xinran Liu, Yifan Bai, Zheng Pan, Mu Xu, and Xing Wei. Futuresightdrive: Thinking visually with spatio-temporal cot for autonomous driving. *arXiv preprint arXiv:2505.17685*, 2025b.

Daochen Zha, Zaid Pervaiz Bhat, Kwei-Herng Lai, Fan Yang, Zhimeng Jiang, Shaochen Zhong, and Xia Hu. Data-centric artificial intelligence: A survey. *ACM Computing Surveys*, 55(9), 2023.

Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. RepoCoder: Repository-level code completion through iterative retrieval and generation. In *EMNLP*, pages 2471–2484, December 2023.

Lunjun Zhang, Arian Hosseini, Hritik Bansal, Mehran Kazemi, Aviral Kumar, and Rishabh Agarwal. Generative verifiers: Reward modeling as next-token prediction, 2025a.

Shenao Zhang, Donghan Yu, Yihao Feng, Bowen Jin, Zhaoran Wang, John Peebles, and Zirui Wang. Learning to reason as action abstractions with scalable mid-training rl, 2025b.

Xuemiao Zhang, Chengying Tu, Can Ren, Rongxiang Weng, Hongfei Yan, Jingang Wang, and Xunliang Cai. Large-scale diverse synthesis for mid-training, 2025c.

Zhongyi Zhang, Zhen Huang, Xianjun Yang, Zhengyuan Liu, Iryna Gurevych, Kai-Wei Chang, Baobao Chang, and Zhiyuan Liu. The LLM data auditor: A survey on data-centric metrics for cross-modal large language models. *arXiv preprint arXiv:2601.17717*, 2026.

Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Yang Yue, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute zero: Reinforced self-play reasoning with zero data, 2025a.

Hao Zhao, Maksym Andriushchenko, Francesco Croce, and Nicolas Flammarion. Long is more for alignment: A simple but tough-to-beat baseline for instruction fine-tuning. In *ICML*, 2024a.

Haokun Zhao, Haixia Han, Jie Shi, Chengyu Du, Jiaqing Liang, and Yanghua Xiao. Cem: A data-efficient method for large language models to continue evolving from mistakes. *arXiv preprint arXiv:2404.08707*, 2024b.

Haokun Zhao, Jinyi Han, Jiaqing Liang, Yanghua Xiao, Xiaojun Meng, and Jiansheng Wei. CDS: Data synthesis method guided by cognitive diagnosis theory. In *ACL-Findings*, pages 8370–8393, July 2025b. ISBN 979-8-89176-256-5.

Jinghan Zhao, Kaizhao Liang, Liangchen Luo, Meiying Wu, Yiqi Zhu, Yue Huang, Ming-Syan Chen, and Yu-Chiang Frank Wang. SWE-ZERO: Self-evolving software agents from zero data. *arXiv preprint arXiv:2604.01496*, 2026a.

Runcong Zhao, Qinglin Zhu, Hainiu Xu, Bin Liang, Lin Gui, and Yulan He. Symbolicthought: Integrating language models and symbolic reasoning for consistent and interpretable human relationship understanding, 2025c.

Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Zican Dong, Yupeng Hou, Beichen Zhang, Yingqian Min, Junjie Zhang, Peiyu Liu, et al. A survey of large language models. *Frontiers of Computer Science*, 20(12):2012627, 2026b.

Xinping Zhao, Dongfang Li, Yan Zhong, Boren Hu, Yibin Chen, Baotian Hu, and Min Zhang. SEER: Self-aligned evidence extraction for retrieval-augmented generation. In *EMNLP*, pages 3027–3041, 2024c.

Xufeng Zhao, Mengdi Li, Wenhao Lu, Cornelius Weber, Jae Hee Lee, Kun Chu, and Stefan Wermter. Enhancing zero-shot chain-of-thought reasoning in large language models through logic. In *LREC-COLING*, pages 6144–6166, May 2024d.

Junhao Zheng, Xidi Cai, Shengjie Qiu, and Qianli Ma. Spurious forgetting in continual learning of language models. In *ICLR*, 2025a.

Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. minif2f: a cross-system benchmark for formal olympiad-level mathematics. In *International Conference on Learning Representations*, 2022.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine, editors, *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023.

Weihua Zheng, Xin Huang, Zhengyuan Liu, Tarun Kumar Vangani, Bowei Zou, Xiyan Tao, Yuhao Wu, Ai Ti Aw, Nancy F. Chen, and Roy Ka-Wei Lee. Adamcot: Rethinking cross-lingual factual reasoning through adaptive multilingual chain-of-thought, 2025b.

Qihuang Zhong, Liang Ding, Juhua Liu, Bo Du, and Dacheng Tao. ROSE doesn't do that: Boosting the safety of instruction-tuned large language models with reverse prompt contrastive decoding. In *ACL-Findings*, pages 13721–13736, August 2024.

Chunting Zhou, Pengfei Liu, Puxin Xu, Srinivasan Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, et al. LIMA: less is more for alignment. In *NeurIPS*, 2023.

Fan Zhou, Zengzhi Wang, Nikhil Ranjan, Zhoujun Cheng, Liping Tang, Guowei He, Zhengzhong Liu, and Eric P. Xing. Megamath: Pushing the limits of open math corpora, 2025a.

Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. In *The Twelfth International Conference on Learning Representations*, 2024.

Xiaofeng Zhou, Heyan Huang, and Lizi Liao. Debate, reflect, and distill: Multi-agent feedback with tree-structured preference optimization for efficient language model enhancement. In *ACL-Findings*, pages 9122–9137, July 2025b. ISBN 979-8-89176-256-5. doi: 10.18653/v1/2025.findings-acl.475.

Xuanhe Zhou, Junxuan He, et al. A survey of LLM × DATA. *arXiv preprint arXiv:2505.18458*, 2025c.

Qihao Zhu, Daya Guo, Zhihong Shao, Dejian Yang, Peiyi Wang, Runxin Xu, Y Wu, Yukun Li, Huazuo Gao, Shirong Ma, et al. Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence. *arXiv preprint arXiv:2406.11931*, 2024.

Yiqi Zhu, Apurva Gandhi, and Graham Neubig. Training versatile coding agents in synthetic environments. *arXiv preprint arXiv:2512.12216*, 2025.

Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, et al. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. In *ICLR*, 2025.

Ingo Ziegler, Abdullatif Köksal, Desmond Elliott, and Hinrich Schütze. Craft your dataset: Task-specific synthetic dataset generation through corpus retrieval and augmentation. In *TACL*, 2025.